



ALPHA COLLEGE OF ENGINEERING
THIRUMAZHISAI, CHENNAI

(Approved by AICTE and Affiliated to Anna University)

PROJECT DOCUMENTATION

EventForce Management System

Salesforce CRM Implementation for Event Planner Solutions

Item	Details
Project Title	EventForce Management System – Salesforce Implementation
Team ID	6ab4d4f2b5170a9b58b51c5d
Team Lead	Ashvitha V (ashvithavaradhan@gmail.com)
Team member	Sowmiya A (sowmiyashanthi71006@gmail.com)
Team member	Saranya S (saranyasaranya369369@gmail.com)
Team member	Vaishnavi V (vaishnavi1512v@gmail.com)
Team member	Nivetha T (nivethathiyagu2006@gmail.com)

Table of Contents

1. INTRODUCTION	3
1.1 Project Overview	3
1.2 Purpose	3
2. IDEATION PHASE	5
2.1 Problem Statement (Customer Problem Statement Template)	5
2.2 Empathy Map Canvas	7
2.3 Brainstorming & Idea Prioritization	8
3. REQUIREMENT ANALYSIS PHASE	9
3.1 Customer Journey Map	9
3.2 Solution Requirements	10
3.3 Data Flow Diagram	11
3.4 Technology Stack	12
4. PROJECT DESIGN PHASE	13
4.1 Problem–Solution Fit	13
4.2 Proposed Solution	15
4.3 Solution Architecture	15
5. PROJECT PLANNING & SCHEDULING	17
5.1 Project Planning	17
6. PROJECT DEVELOPMENT PHASE	19
6.1 Phase 1: Requirement Analysis & Planning	19
6.2 Phase 2: Backend Development & Configurations	20
6.3 Phase 3: UI/UX Development & Customization	34
6.4 Phase 4: Data Migration, Testing & Security	37
6.5 Phase 5: Deployment, Documentation & Maintenance	41
7. FUNCTIONAL & PERFORMANCE TESTING	43
8. ADVANTAGES & DISADVANTAGES	44
9. CONCLUSION	44
10. FUTURE SCOPE	45
11. APPENDIX	46

EVENTFORCE MANAGEMENT SYSTEM- SALESFORCE IMPLEMENTATION

User Story:

The EventForce Management System is designed to provide a centralized Salesforce-based solution for managing event planning operations, including client bookings, event scheduling, venue reservations, vendor coordination, cancellation approvals, reminders, and feedback collection. The system enables event teams to manage Events, Clients, Vendors, Venues, and Feedback using a structured Salesforce data model with appropriate relationships. Automated Flows handle event reminders and process updates, while an Approval Process manages event cancellation requests. Apex Classes and Triggers support advanced business logic such as venue availability updates and prevention of double bookings. A Lightning App, Reports, and Dashboards provide real-time visibility into event schedules, vendor activities, and operational performance. This initiative improves event management efficiency, booking coordination, data accuracy, communication, operational visibility, and customer experience.

1. INTRODUCTION

1.1 Project Overview

The EventForce Management System is a Salesforce-based CRM solution developed to centralize and streamline event planning and management operations. The system manages clients, events, venues, vendors, feedback, and event cancellations within a structured Salesforce platform.

The solution uses custom objects, relationships, validation rules, Flows, Approval Processes, Apex Triggers and Classes, Reports, and Dashboards to automate key business processes. It supports venue availability management, prevention of double bookings, automated event reminders, cancellation approvals, feedback management, and operational reporting.

The project aims to reduce manual effort, improve booking coordination and data accuracy, enhance communication with clients and operational teams, and provide better visibility into event activities through a centralized Salesforce platform.

1.2 Purpose

The purpose of the EventForce Management System is to provide a centralized Salesforce platform for managing and automating event planning operations. The system is designed to:

- Centralize client, event, venue, vendor, and feedback information.
- Streamline event booking and venue reservation processes.
- Prevent double booking of venues through automated Apex logic.
- Automate event reminders and cancellation approval processes.
- Improve coordination between clients, event coordinators, vendors, and venue managers.
- Provide reports and dashboards for better operational visibility and decision-making.

- Reduce manual processes, improve data accuracy, and enhance the overall client experience.

Expected Business Value

Business Outcome	Target
Reduction in manual processes	60%
Improvement in booking efficiency	40%
Client satisfaction	Improved through timely reminders and communications
Decision making	Data-driven insights through integrated reporting and analytics

2. IDEATION PHASE

2.1 Problem Statement (Customer Problem Statement Template)

PS	I am (Customer)	I'm trying to	But	Because	Which makes me feel
PS-1	Event Operations Manager	manage event planning operations efficiently	client, event, venue, and vendor information is difficult to manage centrally	event management activities require coordination across multiple areas	frustrated and concerned
PS-2	Event Coordinator	create and manage event bookings efficiently	event details, client information, and venue reservations require multiple manual activities	there is no centralized platform for managing the complete event lifecycle	overwhelmed and frustrated
PS-3	Venue Manager	manage venue availability accurately	the same venue may be assigned to multiple events	venue bookings need proper availability tracking and double-booking prevention	concerned and uncertain
PS-4	Vendor Manager	coordinate vendors for different events	vendor details and event assignments need to be managed separately	events can require multiple vendors and services	overwhelmed and concerned
PS-5	Event Coordinator	communicate event information and reminders on time	reminders have to be handled manually	there is a need to notify clients before upcoming events	stressed and concerned
PS-6	Event Administrator	manage event cancellation requests systematically	cancellation requests require approval and status updates	cancellation decisions need a controlled approval process	frustrated and uncertain
PS-7	Event Coordinator	collect and manage client feedback	feedback information needs to be maintained along with event and client details	feedback is related to both events and clients	concerned about customer satisfaction
PS-8	Event Operations Manager	monitor event operations and performance	operational information is not easily visible in one place	event data needs to be organized into reports and dashboards	concerned and less confident in decision-making

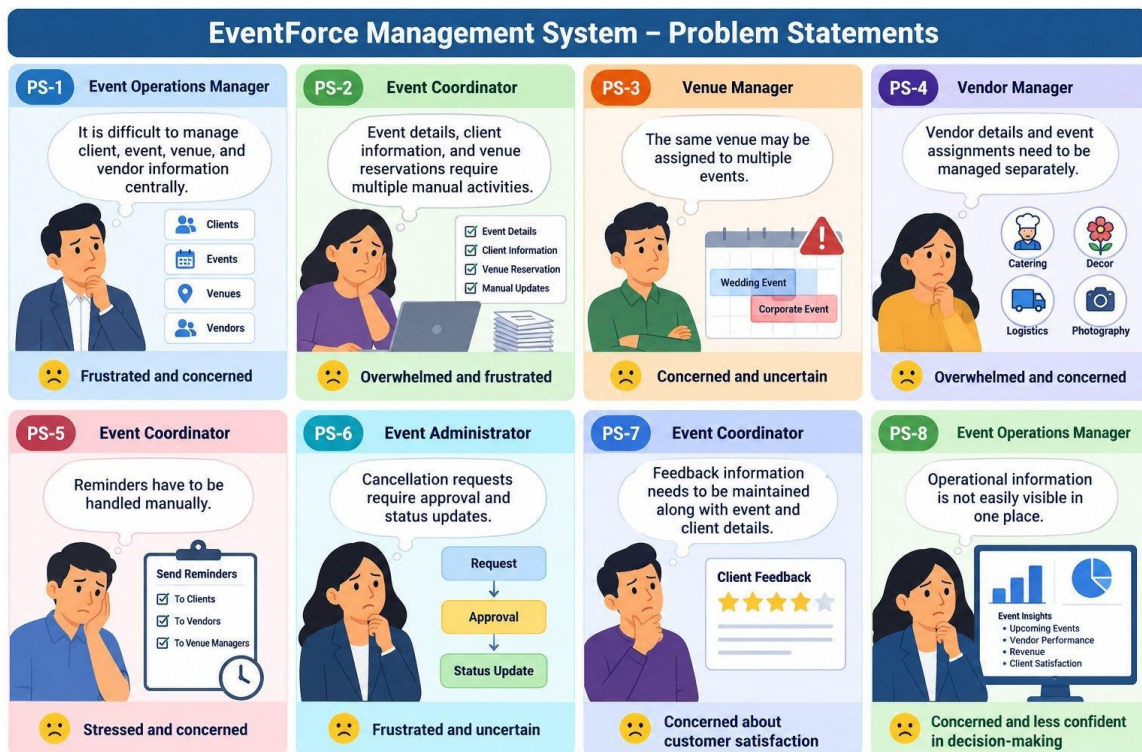


Figure 2.1 – Customer Problem Statements

2.2 Empathy Map Canvas

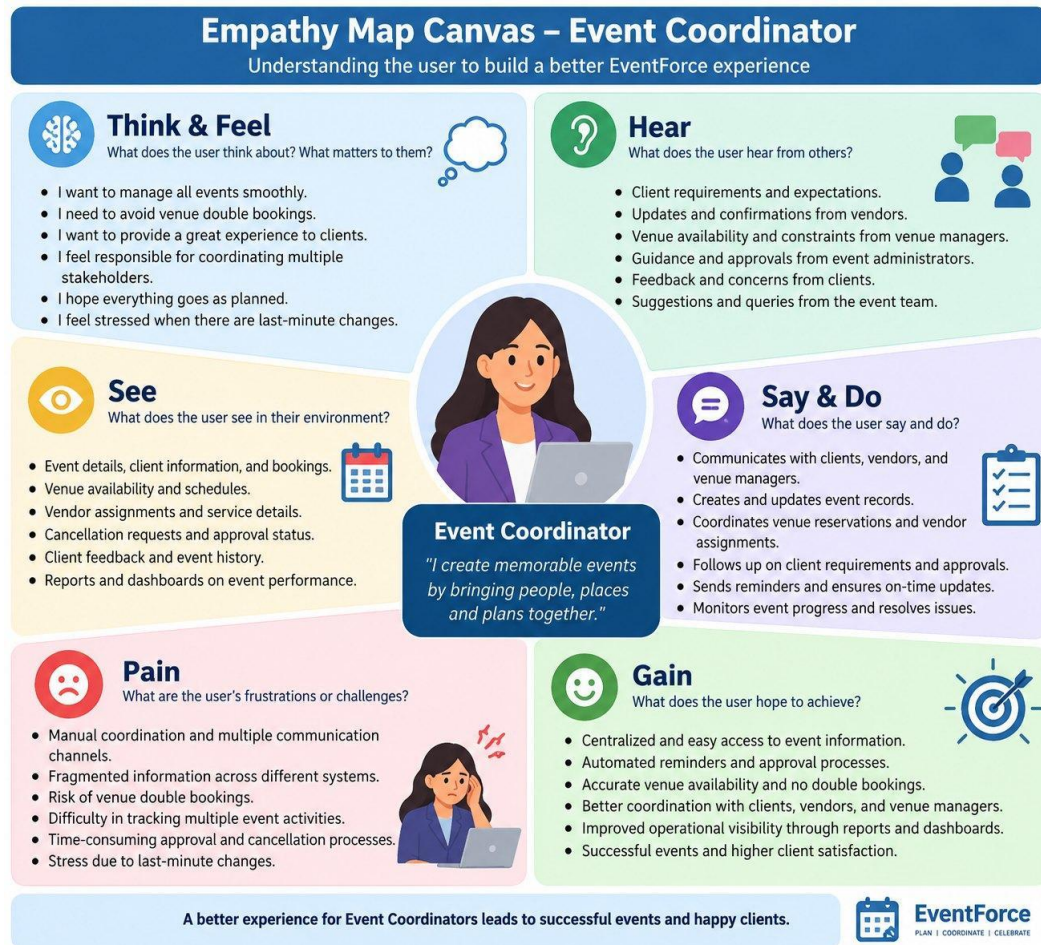


Figure 2.2 – Empathy Map Canvas: Event Coordinator

User Persona: Event Coordinator

Role: Event Coordinator

Primary Responsibility: Manage event bookings, client requirements, venue reservations, vendor coordination, reminders, and event status updates.

- **Think & Feel:** Wants to manage events smoothly, avoid booking conflicts, and provide a reliable experience to clients.
- **Hear:** Receives requirements and updates from clients, vendors, venue managers, and event administrators.
- **See:** Event details, client information, venue availability, vendor assignments, cancellations, and feedback.
- **Say & Do:** Coordinates with stakeholders, updates event records, manages bookings, follows up on requirements, and monitors event progress.
- **Pain:** Manual coordination, fragmented information, venue double-booking risks, and difficulty tracking multiple event activities.
- **Gain:** Centralized event information, automated reminders and processes, accurate venue availability, better coordination, and improved operational visibility.

2.3 Brainstorming & Idea Prioritization



Figure 2.3 – Brainstorming & Idea Prioritization

Ideas Considered

Idea	Approach	Key Limitation
Manual Registers	Maintain event, client, venue, and vendor details manually	Time-consuming and difficult to maintain
Spreadsheet Tracking	Track bookings and event information using spreadsheets	Limited automation and real-time coordination
Standalone Portal	Develop a dedicated portal for event management	Higher development and maintenance effort
Salesforce CRM	Centralize event operations on a Salesforce platform	Requires proper configuration and implementation

Final Selection

Salesforce CRM selected due to:

- **Automation** – Automates reminders, approvals, status updates, and business processes.
- **Scalability** – Supports expanding event operations, users, records, and business requirements.
- **Real-Time Visibility** – Provides centralized data, reports, and dashboards for operational monitoring.

Decision: Salesforce CRM provides the best fit for creating a centralized, automated, and scalable EventForce Management System.

3. REQUIREMENT ANALYSIS PHASE

3.1 Customer Journey Map

Stage	Customer Journey Stage	What Happens
1	Event Enquiry & Client Registration	Client requirements and contact details are captured in Salesforce.
2	Event Planning & Booking	Event details such as event type, date, budget, and status are created and managed.
3	Venue Selection & Reservation	Suitable venues are identified, availability is checked, and reservations are managed while preventing double bookings.
4	Vendor Coordination	Vendors and required services are assigned to events and their availability/status is coordinated.
5	Event Confirmation & Communication	Confirmed events trigger automated communication, including reminders sent to clients before the event.
6	Event Execution & Monitoring	Event progress and operational information are monitored through Salesforce records, reports, and dashboards.
7	Cancellation & Approval	Cancellation requests are submitted through the approval process, with appropriate status updates and notifications.
8	Feedback & Closure	Client feedback is captured after the event, and completed events are updated for operational tracking and future analysis.



Figure 3.1 – Customer Journey Map

3.2 Solution Requirements

Functional Requirements

FR No	Requirement
FR-1	Event management
FR-2	Client management
FR-3	Venue management
FR-4	Vendor management
FR-5	Event–vendor assignment
FR-6	Venue availability & double-booking prevention
FR-7	Automated event reminders
FR-8	Event cancellation approval
FR-9	Feedback management
FR-10	Reports & dashboards
FR-11	Event status automation
FR-12	Data migration through Data Import Wizard

Non-Functional Requirements

NFR	Description
Usability	Lightning App for easy event management and navigation
Security	Profile-, role-, and permission-based access control
Reliability	Accurate automation, approvals, and venue booking validation
Performance	Efficient processing of event and operational data
Availability	Cloud-based Salesforce access for authorized users
Scalability	Supports growing events, clients, vendors, venues, and users
Maintainability	Flows, Apex, validation rules, and security can be monitored and updated
Data Integrity	Validation and controlled data migration to maintain accurate records

3.3 Data Flow Diagram

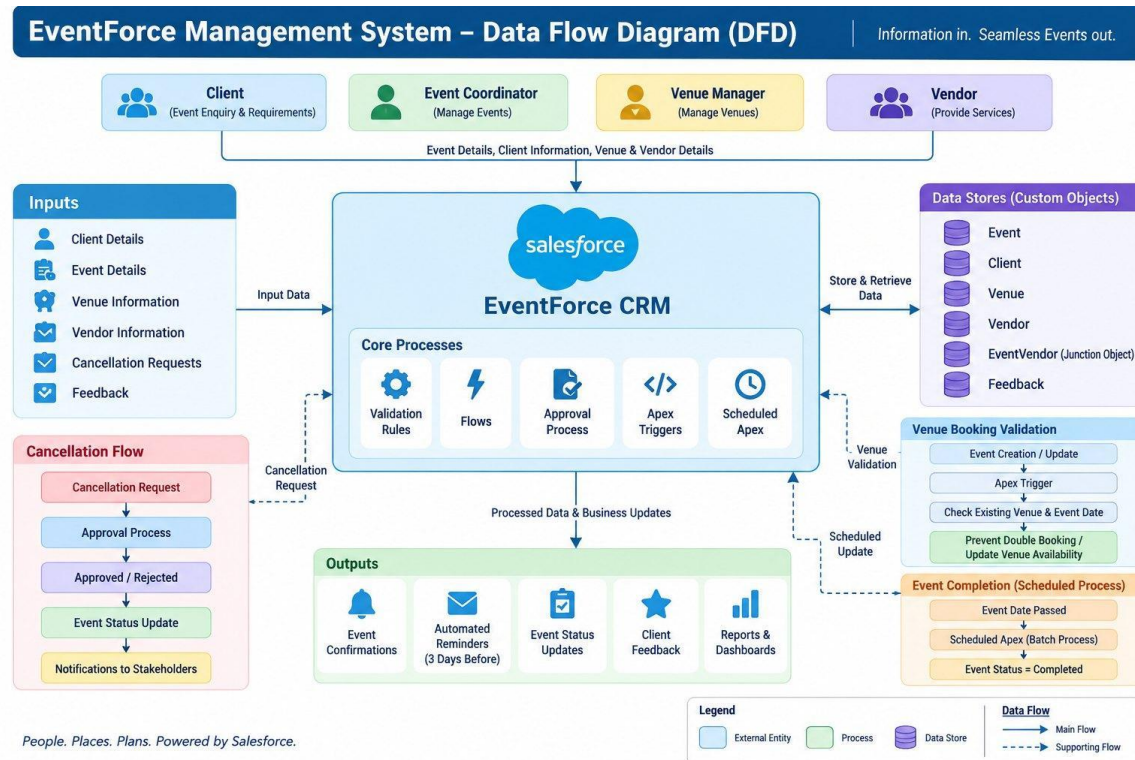


Figure 3.2 – Data Flow Diagram (DFD)

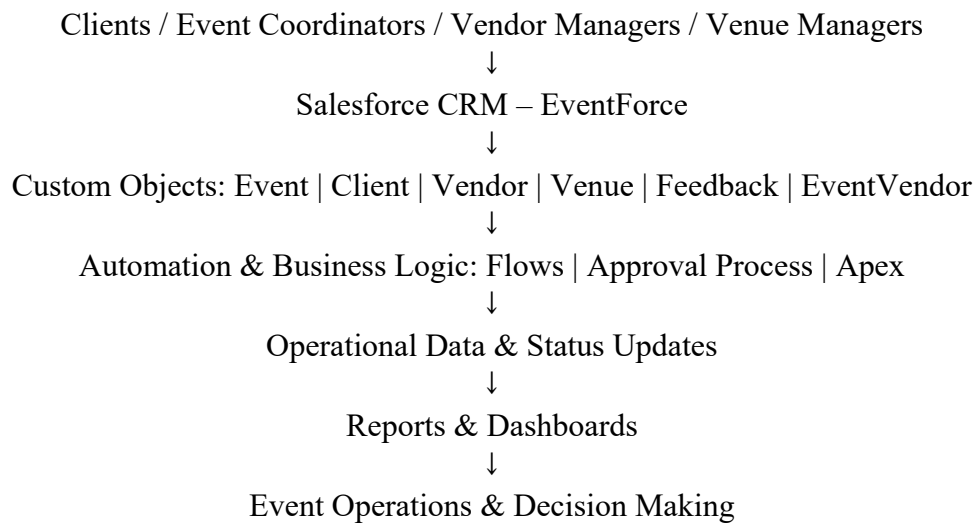
Flow:



Supporting Flows

- **Cancellation Request:** Event Coordinator → Cancellation Request → Approval Process → Approved / Rejected → Event Status Update → Notifications
- **Venue Booking Validation:** Event → Apex Trigger → Venue Status Helper → Check Existing Venue & Event Date → Prevent Double Booking / Update Venue Availability
- **Event Completion:** Event Date Passed → Scheduled Apex → Batch Process → Event Status = Completed

Simple DFD Representation



3.4 Technology Stack

Layer	Technology
Platform	Salesforce CRM / Developer Edition
UI	Salesforce Lightning App
Database	Salesforce Custom Objects
Business Logic	Salesforce Flows, Formula Fields, Validation Rules
Approval	Salesforce Approval Process
Automation	Record-Triggered Flows, Apex Triggers
Processing	Batch Apex, Scheduled Apex
Security	Profiles, Roles, Permission Sets, Sharing Rules, OWD
Reporting	Reports & Dashboards

4. PROJECT DESIGN PHASE

4.1 Problem–Solution Fit

The EventForce Management System provides a strong fit between identified event-management problems and Salesforce capabilities. The solution combines custom data structures, automation, Apex business logic, approval processes, security controls, and reporting to address the operational challenges of event planning and coordination.

Problem	Solution – EventForce Salesforce System	Fit / Benefit
Client, event, venue, and vendor information is difficult to manage centrally	Custom Salesforce objects for Event, Client, Venue, Vendor, Feedback, and EventVendor	Centralized and structured event data
Event booking requires multiple manual activities	Salesforce Flows and automated processes	Reduces manual effort and improves process efficiency
Risk of venue double booking	Apex Trigger and VenueStatusHelper	Validates venue availability and prevents double bookings
Vendor coordination is difficult to track	EventVendor junction object and Vendor records	Supports event–vendor assignment and coordination
Clients may not receive timely event reminders	3-Day Reminder Flow	Automates reminder communication before confirmed events
Cancellation requests require controlled processing	Event_Cancellation_Process Approval Process	Provides structured approval and status updates
Client feedback needs to be connected with events	Feedback object with Event and Client relationships	Enables organized feedback management
Operational information is difficult to monitor	Reports and EventForce Operations Dashboard	Provides better visibility for monitoring and decisions
Different users require different levels of access	Profiles, Roles, Permission Sets, OWD, and Sharing Rules	Provides controlled and secure data access



Figure 4.1 – Problem–Solution Fit

4.2 Proposed Solution

Parameter	Description
Problem	Manual and fragmented event management
Solution	Salesforce CRM
Innovation	Automated event reminders, venue validation, and cancellation approvals
Impact	Faster event coordination and improved operational visibility
Business Model	Centralized event planning and management
Scalability	Supports growing events, clients, vendors, venues, and users

4.3 Solution Architecture

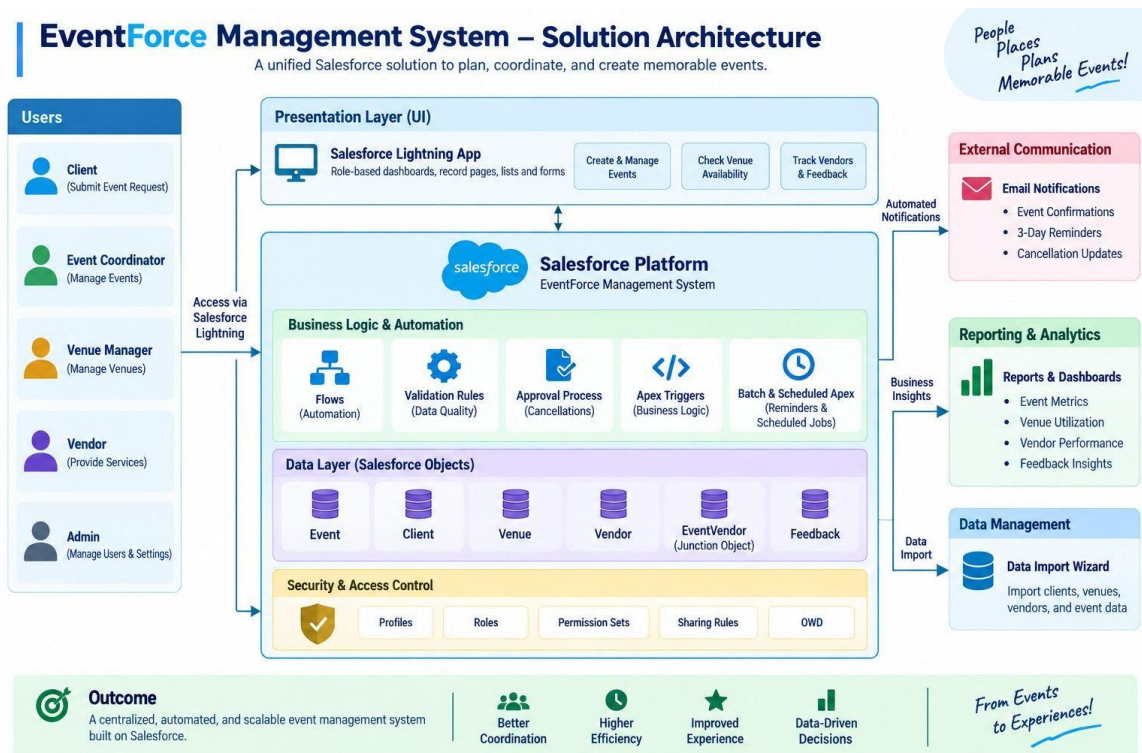


Figure 4.2 – Solution Architecture

Components:

- Event Object
- Client Object
- Vendor Object
- Venue Object
- EventVendor Junction Object
- Feedback Object
- Validation Rules
- Formula Fields
- Flows
- Approval Process
- Apex Triggers
- Apex Classes

- Batch Apex
- Scheduled Apex
- Profiles & Roles
- Permission Sets & Sharing Rules
- Data Import Wizard
- Lightning App
- Reports
- Dashboards

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

- Agile methodology
- Epics → Stories → Story Points
- Sprint-based execution
- Velocity calculation used for estimation

Product Backlog, Sprint Schedule, and Estimation

Sprint	Functional Requirement (Epic)	User Story No.	User Story / Task	Story Points	Priority	Team Member
Sprint-1	Event Management	US-01	As an Event Coordinator, I want to create events with type, date, and status so that event information is standardized.	3	High	Ashvitha V
Sprint-1	Client & Venue Management	US-02	As an Event Coordinator, I want to associate a client and venue with an event so that coordination is centralized.	5	High	Sowmiya A
Sprint-2	Vendor Management	US-03	As a Vendor Manager, I want to assign multiple vendors to an event so that services can be coordinated.	5	High	Saranya S
Sprint-2	Notifications & Automation	US-04	As a Client, I want timely event reminders so that I know when my event is approaching.	3	High	Vaishnavi V
Sprint-3	Approval Management	US-05	As an Event Coordinator, I want cancellation requests to follow an approval process so that cancellations are controlled.	5	High	Nivetha T
Sprint-3	Venue Management & Apex	US-06	As a Venue Manager, I want double bookings prevented so that venue schedules remain accurate.	8	High	Ashvitha V
Sprint-4	Security & Access Control	US-07	As an Event Administrator, I want role-based access so that users see only appropriate information.	5	High	Sowmiya A
Sprint-4	Reports & Dashboards	US-08	As Management, I want reports and dashboards so that event operations can be monitored.	5	Medium	Saranya S

Project Tracker, Velocity & Burndown Chart

Sprint	Total Story Points	Duration	Sprint Start Date	Sprint End Date (Planned)	Story Points Completed	Sprint Release Date (Actual)
Sprint-1	20	6 Days	24 Oct 2022	29 Oct 2022	20	29 Oct 2022
Sprint-2	20	6 Days	31 Oct 2022	05 Nov 2022		

Sprint	Total Story Points	Duration	Sprint Start Date	Sprint End Date (Planned)	Story Points Completed	Sprint Release Date (Actual)
Sprint-3	20	6 Days	07 Nov 2022	12 Nov 2022		
Sprint-4	20	6 Days	14 Nov 2022	19 Nov 2022		

Velocity: the team planned 20 story points per sprint (80 story points over 4 sprints of 6 days each), giving an average velocity of 20 story points per sprint, or about 3.33 story points per day ($20 \div 6$).

6. PROJECT DEVELOPMENT PHASE

6.1 Phase 1: Requirement Analysis & Planning

Build an EventForce Management System in Salesforce to help event planners manage client bookings, vendor coordination, venue reservations, and feedback collection — all within a single CRM platform. The system automates key processes such as event scheduling, cancellation approvals, and client reminders, while providing real-time visibility through reports and dashboards.

This phase defined what the system must do, who will use it and in what order it would be built. The solution was scoped as a single Salesforce application that manages client bookings, vendor coordination, venue reservations and feedback, automates event scheduling, cancellation approvals and client reminders, and exposes real-time reports and dashboards.

6.1.1 Functional Scope

- Custom objects: Event, Client, Vendor, Venue, Feedback and Event Vendor (Junction Object).
- Relationship modeling: Lookup, Master-Detail and Many-to-Many (Event ↔ Vendor).
- Automation using Flows for reminders, feedback notifications and approval updates.
- Validation Rules and Formula Fields for accurate data entry and event-budget calculation.
- Approval Processes for event cancellation requests with email alerts to coordinators and clients.
- Profiles, Roles and Permission Sets to manage user-level access and security.
- Reports and Dashboards for event summary, vendor performance and upcoming-event insights.

6.1.2 Stakeholder Mapping

Stakeholders were mapped to Salesforce roles and profiles as shown below.

Stakeholder	Salesforce Role	Salesforce Profile	Primary Needs
Event Administrator	Event Admin	Event Admin	Full system oversight, security and monitoring
Event Coordinator	Event Coordinator	Event Coordinator	Event scheduling, client communication, approvals
Vendor Manager	Vendor Manager	Vendor Manager	Vendor records and assigned services
Client	Client	Client Profile	View own events, submit feedback

6.1.3 Execution Roadmap

Step	Activity	Outcome
1	Object & Field Setup	Event, Client, Vendor, Venue, Feedback and Event Vendor objects created
2	Automation Flows	Record-triggered flow for event reminders
3	Validation Rules & Formula Fields	Data accuracy and budget calculations enforced
4	Profiles, Roles, Permission Sets	User access and record visibility configured
5	Approval Process	Event cancellation workflow with notifications
6	Apex Classes & Triggers	Venue availability updates and double-booking prevention
7	Reports & Dashboards	Analytics for upcoming events and performance

6.2 Phase 2: Backend Development & Configurations

The backend phase built the functional engine of the system. Declarative tools (Record-Triggered Flows, Validation Rules, Formula Fields and Approval Processes) automated booking confirmations, reminders and cancellation approvals, while Apex classes and triggers handled advanced logic such as venue availability updates and double-booking prevention. The phase was executed through the milestones described below.

6.2.1 Milestone 1 – Salesforce Developer Account

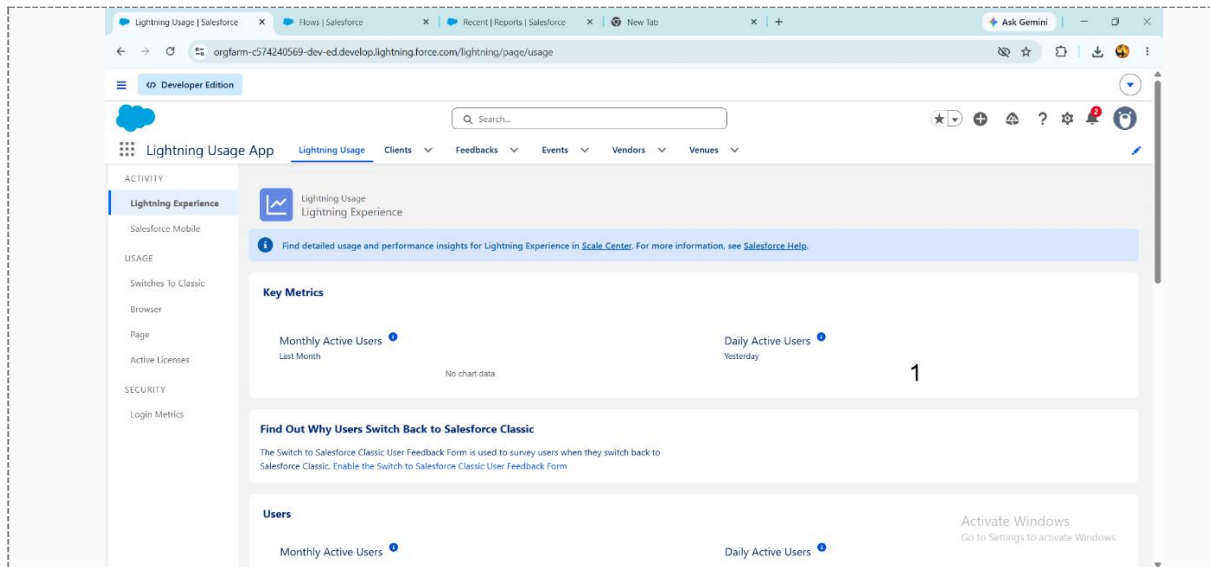
Salesforce is a customer success platform used to sell, service, market, analyze and connect with customers. A free Developer Edition org was created to build and test the entire project.

Activity 1: Creating the Developer Account

1. Open <https://developer.salesforce.com/signup>.
2. Enter First Name, Last Name and Email; select Role as Developer; enter the college name as Company; Country as India; and the Postal Code.
3. Enter a unique Username in email format (for example, username@organization.com). It need not be a real mailbox.
4. Click Sign me up.

Activity 2: Account Activation

1. Open the verification email (it may take 5–10 minutes) and click Verify Account.
2. Set a password and answer the security question, then click Change Password.
3. The user is redirected to the Salesforce Setup home page.



6.2.2 Data Architecture

The data model consists of five primary custom objects and one junction object. An object is a database table that stores organization-specific data; it consists of fields (columns) and records (rows).

Object	API Name	Record Name	Purpose
Event	Event__c	Event Name (Text)	Stores event details, status and budget
Client	Client__c	Client Name (Text)	Stores client contact information
Vendor	Vendor__c	Vendor Name (Text)	Stores vendor details and services
Venue	Venue__c	Venue Name (Text)	Stores venue details and availability
Feedback	Feedback__c	Feedback ID (Auto Number, F-{0000})	Stores client ratings and comments
Event Vendor	EventVendor__c	Auto-generated	Junction object linking Event ↔ Vendor

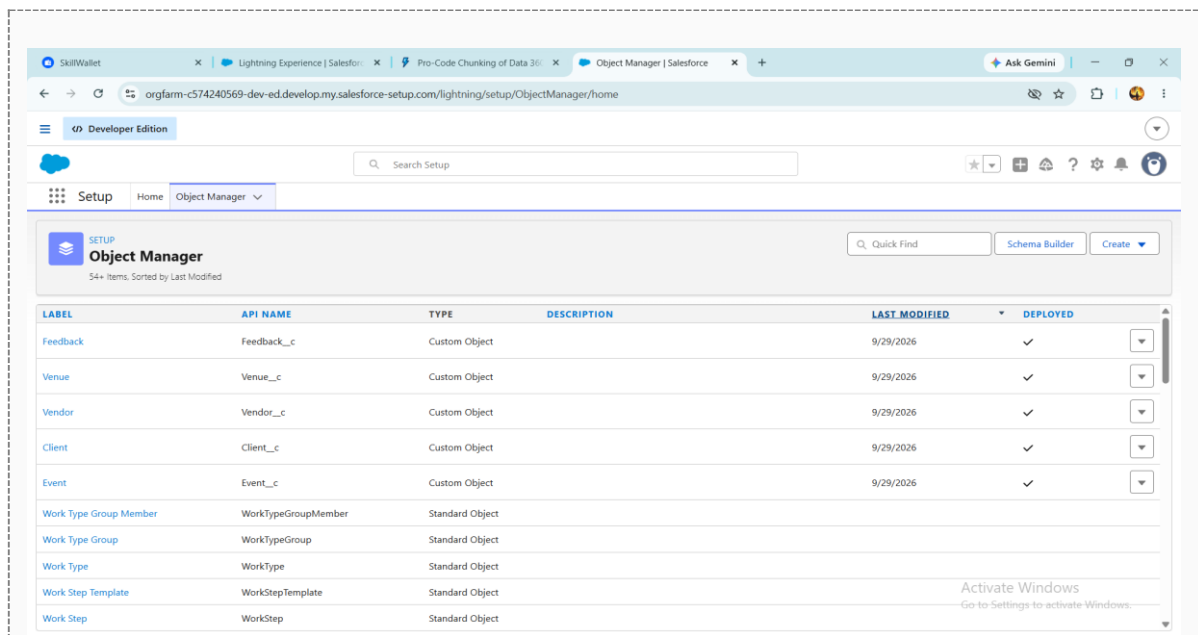
Milestone 2 – Creating Custom Objects

Each object was created from Setup → Object Manager → Create → Custom Object. The Event object was created with the following options:

- Label: Event; Plural Label: Events; Record Name: Event Name; Data Type: Text.
- Optional Features: Allow Reports, Allow Activities and Track Field History.
- Deployment Status: Deployed; Search Status: Allow Search.
- Object Creation Options: Add Notes and Attachments related list to the default page layout.

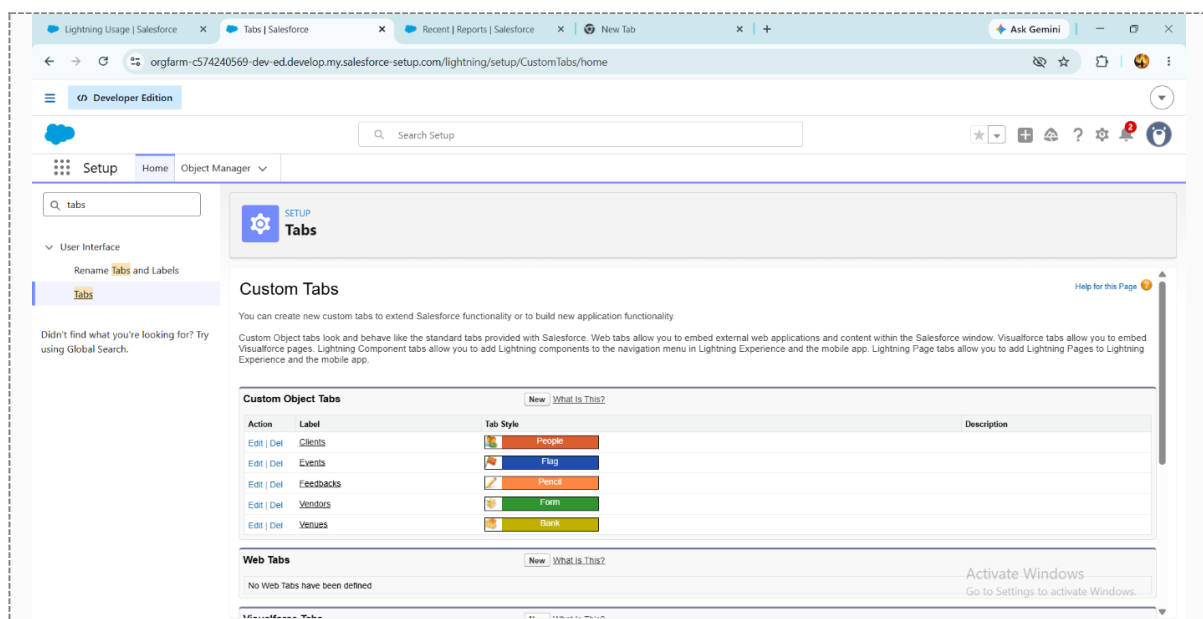
The Client, Vendor and Venue objects were created in the same way (Record Name of Text type, with Allow Reports, Allow Search and the Notes and Attachments related list enabled). The

Feedback object uses an Auto Number record name with display format F-{0000} and a starting number of 0. The Event Vendor junction object was created separately (see Relationships under Section 6.2.2).



Milestone 3 – Custom Tabs

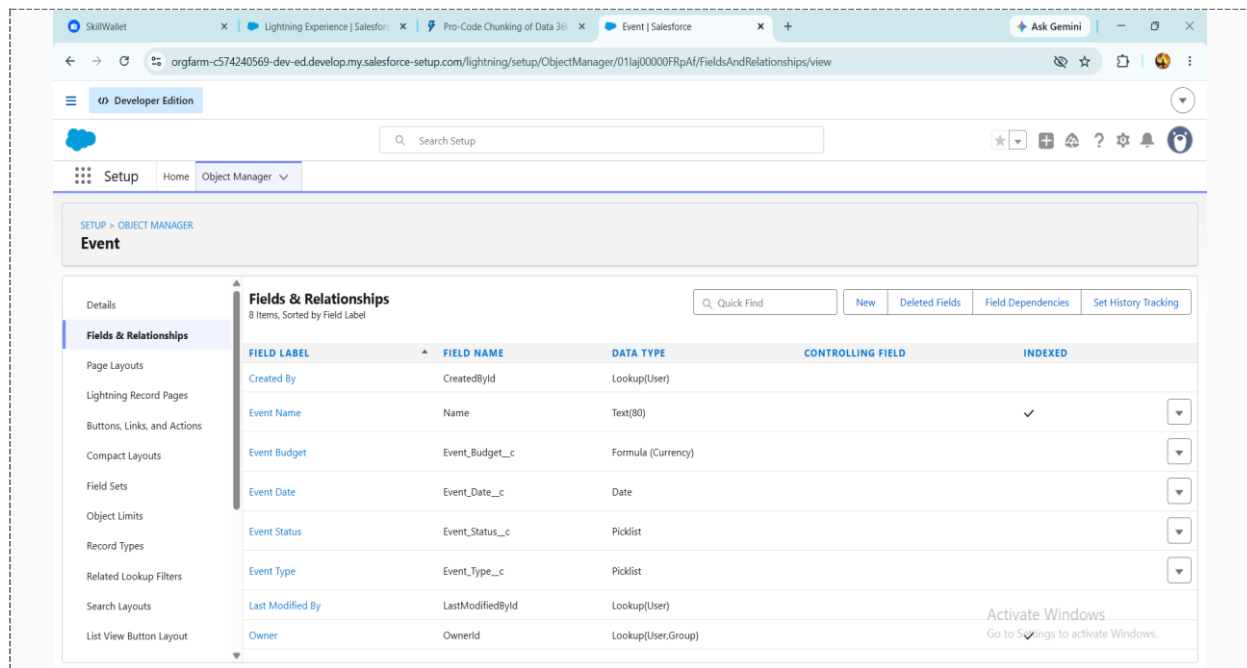
Tabs display object data at a glance. A custom object tab was created for each object using Setup → Tabs → Custom Object Tabs → New. For each of Event, Client, Vendor, Venue and Feedback: select the object, choose a Tab Style icon, and accept the defaults (Next, Next, Save).



Milestone 4 – Fields

Custom fields were added to each object from Object Manager → Fields & Relationships → New, choosing the appropriate data type.

Object	Field Label	Data Type	Values / Notes
Event	Event Type	Picklist	Wedding, Corporate, Birthday, Anniversary, Festival, Concert, Other
Event	Event Date	Date	Date of the event
Event	Event Status	Picklist	Planned, Confirmed, Completed, Pending Cancellation, Canceled, Rejected
Event	Event Budget	Formula (Currency)	Calculated from Event Type (see Section 6.2.3)
Client	Email	Email	Client email address
Client	Phone	Phone	Client contact number
Client	Address	Text Area	Client address
Client	Country	Picklist	India, Philippines, Saudi Arabia, USA, Switzerland, Thailand, Maldives, Dubai, Sri Lanka, Bangkok, Singapore, Myanmar, France, Nepal
Client	City	Picklist	Hyderabad, Mumbai, Delhi, Bangalore, Chennai, Alaminos, Angeles City, Palayan, Quezon City, San Juan, Riyadh, Jeddah, Tayma, Dammam, Medina, New York, Los Angeles, Chicago, Houston, Columbus, Geneva, Basel, Lausanne, Bern, Lucerne
Vendor	Service Type	Multi-Select Picklist	Catering, Decor, Photography, Videography, Lighting, Stage Setup, Makeup Artist, DJ/Music, Transportation, Hosting/Anchor
Vendor	Email	Email	Vendor email address
Vendor	Phone	Phone	Vendor contact number
Vendor	Status	Picklist	Available, Booked, Cancelled
Venue	Address	Text Area	Venue address
Venue	Location	URL	Map link of the venue
Venue	Capacity	Number	Maximum guest capacity
Venue	Availability Status	Picklist	Available, Reserved, Not Available
Feedback	Rating	Picklist	1, 2, 3, 4, 5
Feedback	Comments	Long Text Area	Client comments



Relationships

Relationships connect the objects. Lookup relationships link loosely coupled records, whereas Master-Detail relationships create a tight parent-child dependency. Salesforce does not allow a direct many-to-many relationship, so the Event Vendor junction object with two Master-Detail relationships was created to link Event and Vendor.

Relationship	Type	Field Label
Event → Client	Lookup	Client
Event → Venue	Lookup	Venue
Feedback → Event	Lookup	Event
Feedback → Client	Lookup	Client
EventVendor → Event	Master-Detail	Event
EventVendor → Vendor	Master-Detail	Vendor

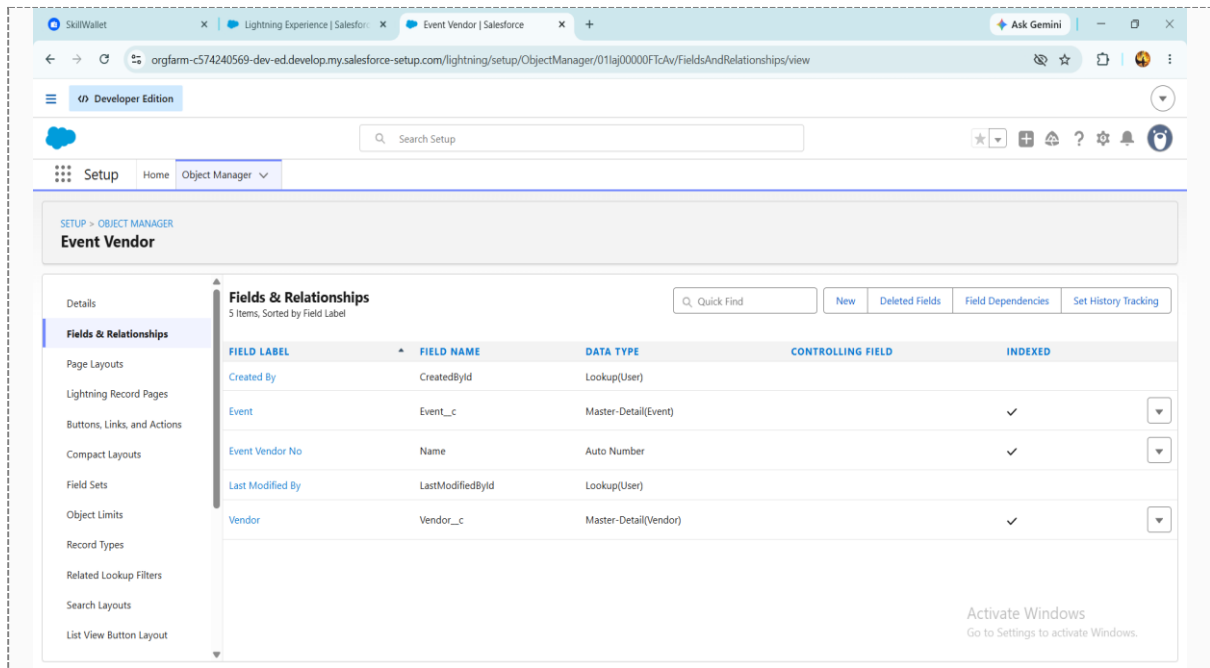
Creating the Lookup Relationships

1. From Setup open Object Manager and select the Event object.
2. Click Fields & Relationships → New → Lookup Relationship → Next.
3. Choose the related object (Venue), click Next, enter the field label and continue with Next, Next, Next and Save.
4. Repeat to add the Client lookup on Event, and the Event and Client lookups on Feedback.

Creating the Many-to-Many Relationship (Event ↔ Vendor)

1. Go to Object Manager → Create → Custom Object. Label: Event Vendor; Object Name: EventVendor__c.

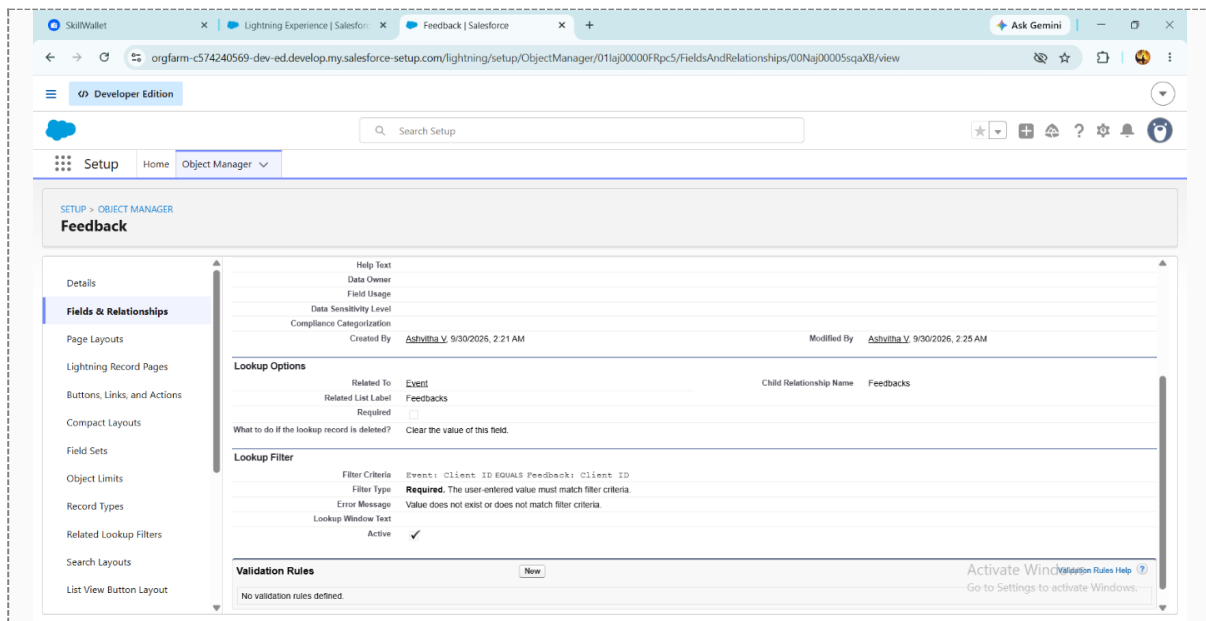
2. On the Event Vendor object click Fields & Relationships → New → Master-Detail Relationship. Choose Event as the related object, label it Event and click Save & New.
3. Create a second Master-Detail Relationship to Vendor with the label Vendor and click Save.



Lookup Filter (Feedback → Event)

Scenario: when a Client is selected on a Feedback record, the Event lookup must show only that client's events. For example, if Ramesh is selected, only Ramesh's events should appear.

1. Open Object Manager → Feedback → Fields & Relationships → Event (Event__c) and click Edit.
2. In the Lookup Filter section click Show Filter Settings.
3. Build the criteria: Event: Client (Client__c) equals Field → Feedback: Client (Client__c).
4. Tick Enable this filter and click Save.

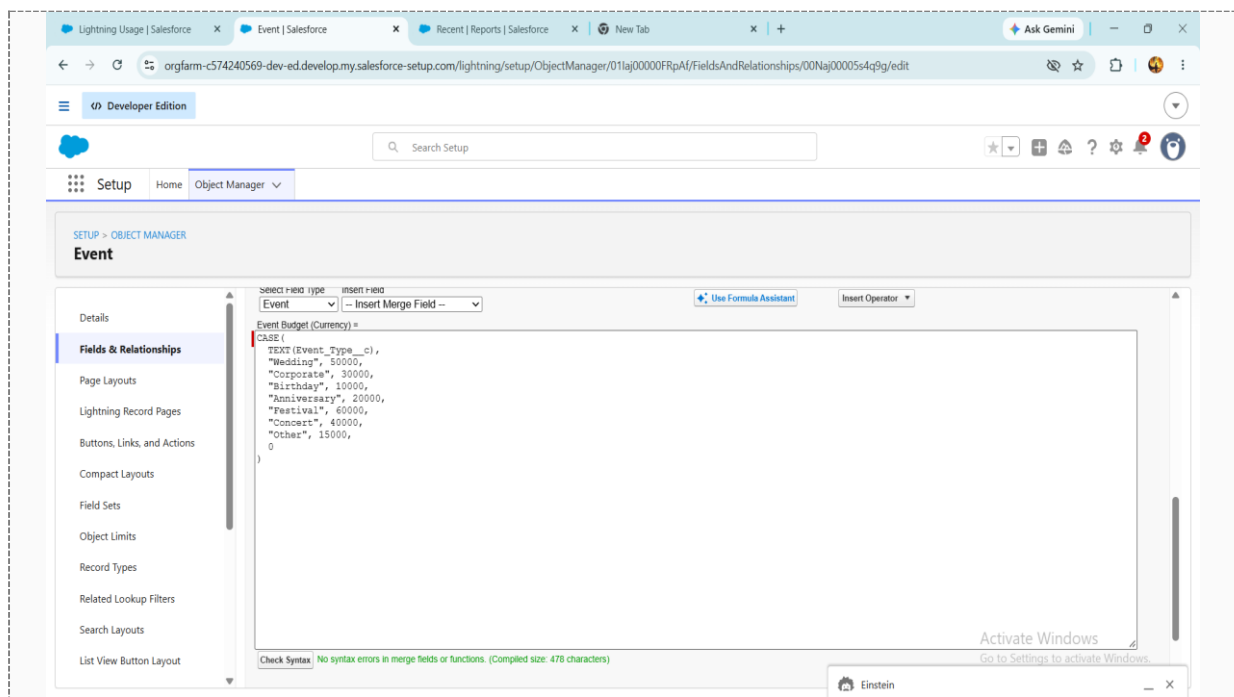


6.2.3 Formula Field – Event Budget

The Event Budget is a Formula field (return type Currency) on the Event object that derives the budget automatically from the Event Type, removing manual entry errors. Steps: Fields & Relationships → New → Formula → Label “Event Budget” → Return type Currency → insert the formula below → Check Syntax → Next, Next, Save.

```
CASE (
  TEXT(Event_Type__c),
  "Wedding", 50000,
  "Corporate", 30000,
  "Birthday", 10000,
  "Anniversary", 20000,
  "Festival", 60000,
  "Concert", 40000,
  "Other", 15000,
  0
)
```

Event Type	Budget
Wedding	50,000
Corporate	30,000
Birthday	10,000
Anniversary	20,000
Festival	60,000
Concert	40,000
Other	15,000

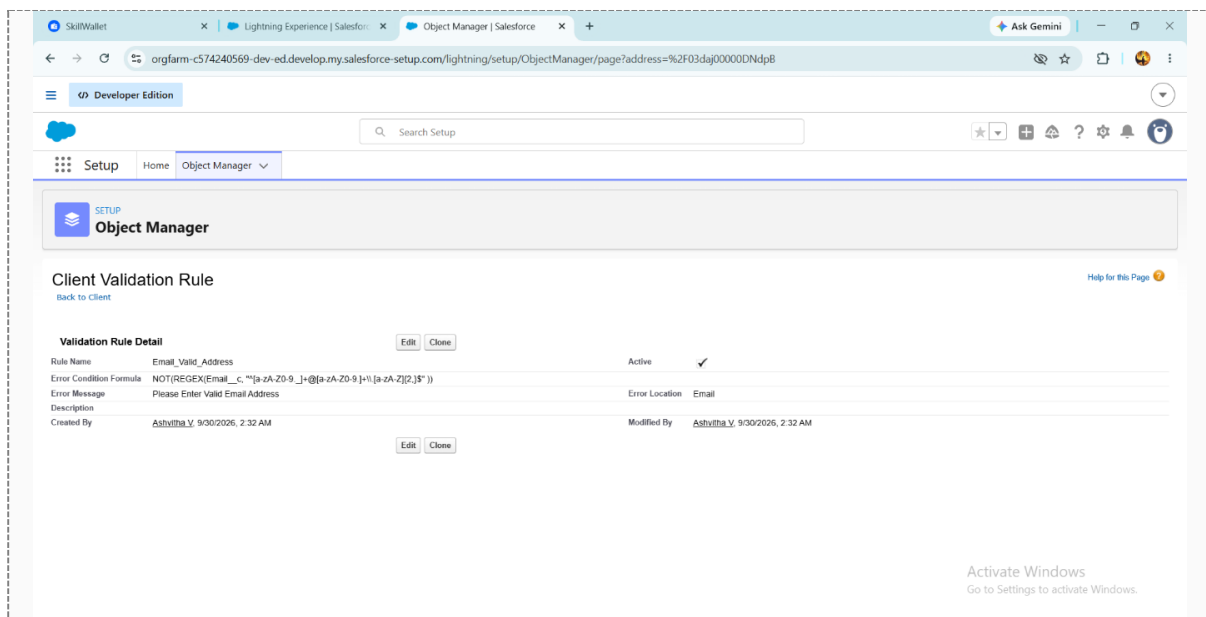


6.2.4 Milestone 5 – Validation Rules

Validation rules run when a user saves a record and block the save if the data does not meet the defined criteria. A validation rule was created on the Client object to ensure that only properly formatted email addresses are stored.

1. Go to Setup → Object Manager → Client → Validation Rules → New.
2. Rule Name: Email_Valid_Address; tick Active.
3. Enter the Error Condition Formula shown below.
4. Error Message: “Please Enter Valid Email Address”; Error Location: Field → Email.
Click Save.

```
NOT (REGEX (Email__c, "^[a-zA-Z0-9._]+@[a-zA-Z0-9]+\.[a-zA-Z]{2,}$"))
```



6.2.5 Milestone 6 – Approval Process (Event Cancellation)

The cancellation approval process ensures that an event is canceled only after manager approval, with automatic notifications to all parties. Note: the Profiles, Roles and Users milestones (Sections 6.4.1–6.4.3) must be completed before implementing the approval process.

Step A – Classic Email Templates

Four Text-type Classic Email Templates were created under Setup → Email Templates → Classic Email Templates → New Template (folder: Unfiled Public Email Templates, encoding UTF-8).

Template Name	Subject	Used For
Event Cancellation Request Notification	Approval Request: Cancel Event {!Event__c.Name}	Approval request submission alert
Email to Client After Cancellation	Event Cancellation Notice – {!Event__c.Name}	Informs client once the cancellation is approved
Approval Notification to Event Owner	Event Cancellation Approved: {!Event__c.Name}	Owner notification on approval
Rejection Notification to Event Owner	Event Cancellation Rejected: {!Event__c.Name}	Owner notification on rejection

Sample Template Body – Event Cancellation Request Notification

```
Hello!

An approval request has been submitted to cancel the event:
- Event Name: {!Event__c.Name}
- Event Date: {!Event__c.Event_Date__c}

Thank you,
EventForce Management System
```

Sample Template Body – Approval Notification to Event Owner

```
Hello {!Event__c.OwnerFullName},

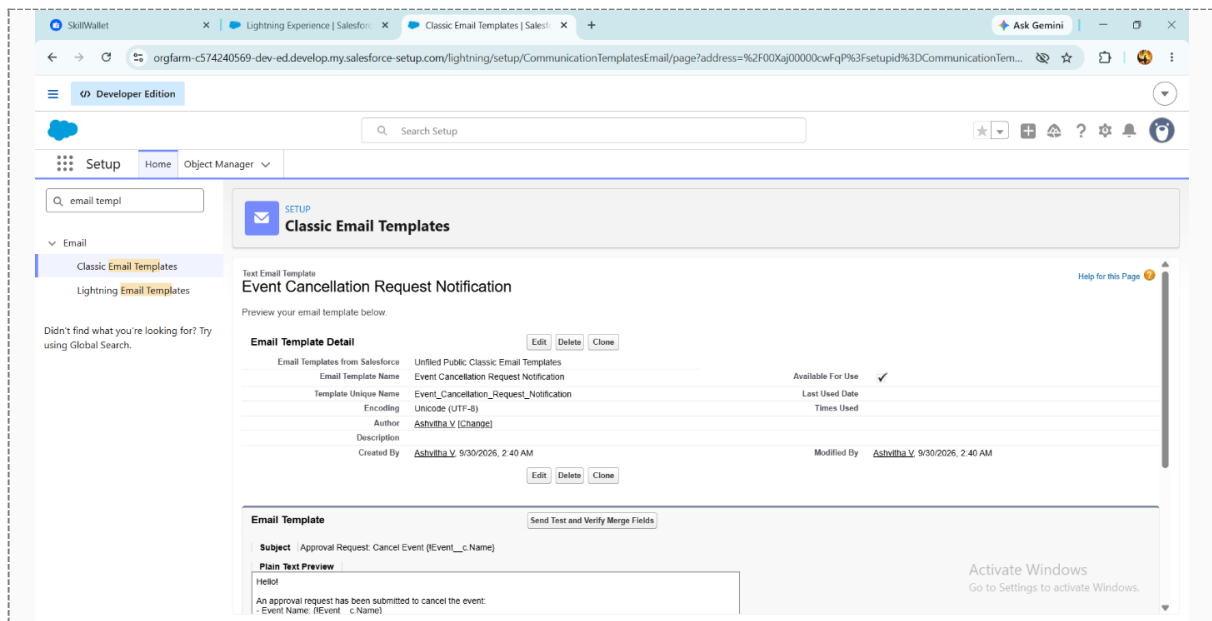
Your cancellation request for the event {!Event__c.Name} has been approved.

Event Date: {!Event__c.Event_Date__c}

The event status has been updated to Canceled.

Thank you,
EventForce Management System
```

The Rejection Notification template follows the same structure, stating that the request has been rejected and that the event status has been reverted to Confirmed. The Email to Client After Cancellation template informs the client that the event (name and date) has been canceled and invites them to contact the team with any questions.



Step B – Creating the Approval Process

1. Setup → Approval Processes → select object Event → Create New Approval Process → Use Standard Setup Wizard.
2. Basic Settings: Name = Event_Cancellation_Process; unique name auto-fills.
3. Entry Criteria: Criteria are met → Event: Event Status equals Pending Cancellation.
4. Select Approver: automatically assign using a user or queue (a specific user / manager of the submitting user; ensure a manager is set on the User record).
5. Email Template: Event Cancellation Request Notification.
6. Approval Page Fields: Event Name, Owner, Client, Last Modified By, Created By, Event Budget, Event Date, Event Status, Event Type, Venue.
7. Initial Submitters: Event Owner. Click Save.

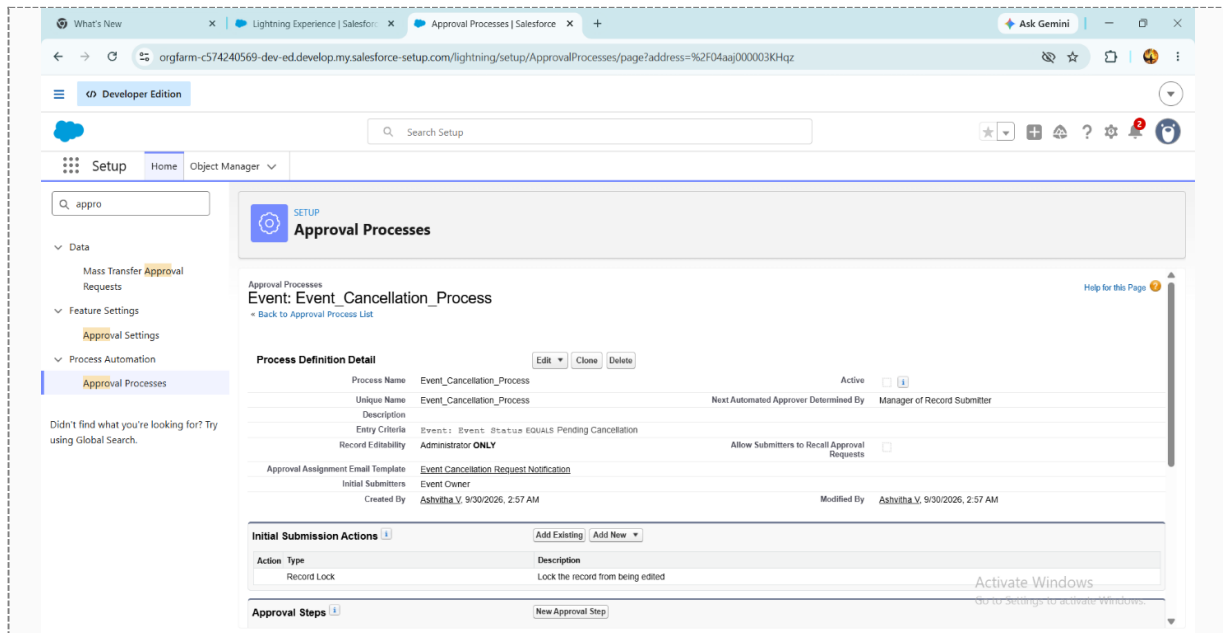
Step C – Actions

Stage	Action Type	Configuration
Initial Submission	Email Alert	Alert – Event Cancellation Request; template: Event Cancellation Request Notification; recipient: Role – Event Coordinator; From: Current User's Email
Final Approval	Field Update	Name: Event Status Update; Field: Event Status; New Value: Canceled
Final Approval	Email Alert	Alert – Cancellation Approved; template: Approval Notification to Event Owner; recipient: Event Owner
Final Rejection	Email Alert	Alert – Cancellation Rejected; template: Rejection Notification to Event Owner; recipient: Event Owner
Final Rejection	Field Update	Name: Event Status To Rejected; Field: Event Status; New Value: Rejected

Step D – Approval Step and Activation

1. Click Add New Step; Step Name: Manager Review; All records should enter this step; click Next.

2. Assign the approver automatically (a specific user / related user) and click Save.
3. Open View Approval Process Detail Page and click Activate.



6.2.6 Milestone 7 – Flow: 3-Day Reminder Email to Client

Scenario: when an Event is created or updated with Event Status = Confirmed and has a client with a valid email address, the system automatically sends a reminder email to the client three days before the Event Date.

Flow Configuration

1. Setup → Flows → New Flow → Record-Triggered Flow → Create.
2. Object: Event; Trigger: A record is created or updated.
3. Condition: Event Status equals Confirmed. Optimize the flow for Actions and Related Records; click Done.
4. Click Start → Add Schedule Path. Path Name: 3_Days_Before; Time Source: Event__c: Event Date; Offset: 3 Days Before.
5. Toolbox → New Resource → Text Template. API Name: Subject; Body: “Reminder: Your Event {!\$Record.Name} is in 3 days” (plain text).
6. Create a second Text Template with API Name Body (content shown below).
7. Under the 3_Days_Before path click + → Action → Send Email. Label: Alert_Client_3Day_Reminder.
8. Recipient Address List: Client Email; CC Recipient Address List: Event Owner.
9. Configure Email Content → Compose Email Content: Subject = Subject resource; Body = Body resource.

10. Click Save, enter Flow Label “Client Reminder - 3 Days Before”, then Save & Activate.

Body Text Template

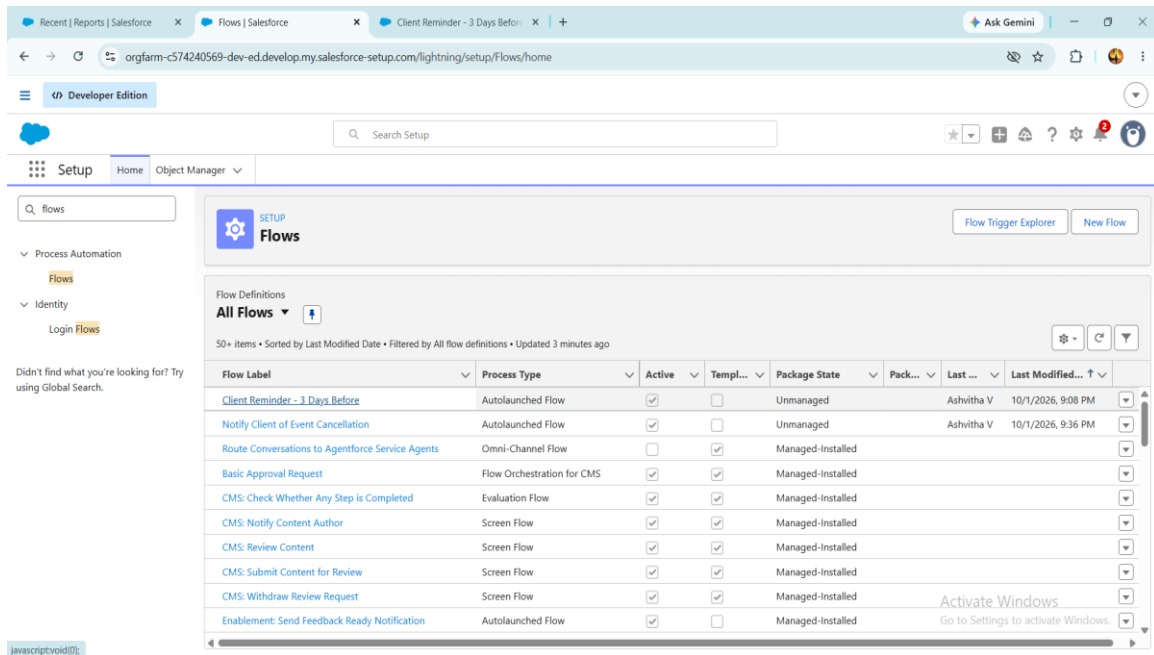
```
Hello {!$Record.Client__r.Name},

This is a reminder that your event "{!$Record.Name}" will take place on
{!$Record.Event_Date__c}.

Venue: {!$Record.Venue__r.Name}
Type: {!$Record.Event_Type__c}

We look forward to seeing you!

- EventForce Team
```



6.2.7 Milestone 8 – Apex Class & Trigger: Venue Status Update

Scenario: when an Event is created or updated with a Venue, the venue’s Availability Status is set to Reserved when the event is Confirmed, and set back to Available when the event is Canceled. The logic is kept in a helper class and invoked from a thin trigger – a recommended best practice.

Apex Class: VenueStatusHelper

Open Gear Icon → Developer Console → File → New → Apex Class → name it VenueStatusHelper, then enter and save (Ctrl+S):

```
public with sharing class VenueStatusHelper {
    public static void updateVenueStatus(List<Event__c> events){
        List<Venue__c> venues = new List<Venue__c>();
        for(Event__c ev : events){
            if(ev.Venue__c != null){
                if(ev.Event_Status__c == 'Confirmed'){
                    venues.add(new Venue__c(Id=ev.Venue__c,
Availability_Status__c='Reserved'));
                } else if(ev.Event_Status__c == 'Canceled'){
                    venues.add(new Venue__c(Id=ev.Venue__c,
Availability_Status__c='Available'));
                }
            }
        }
    }
}
```

```

    }
}
    if(!venues.isEmpty()) update venues;
}
}

```

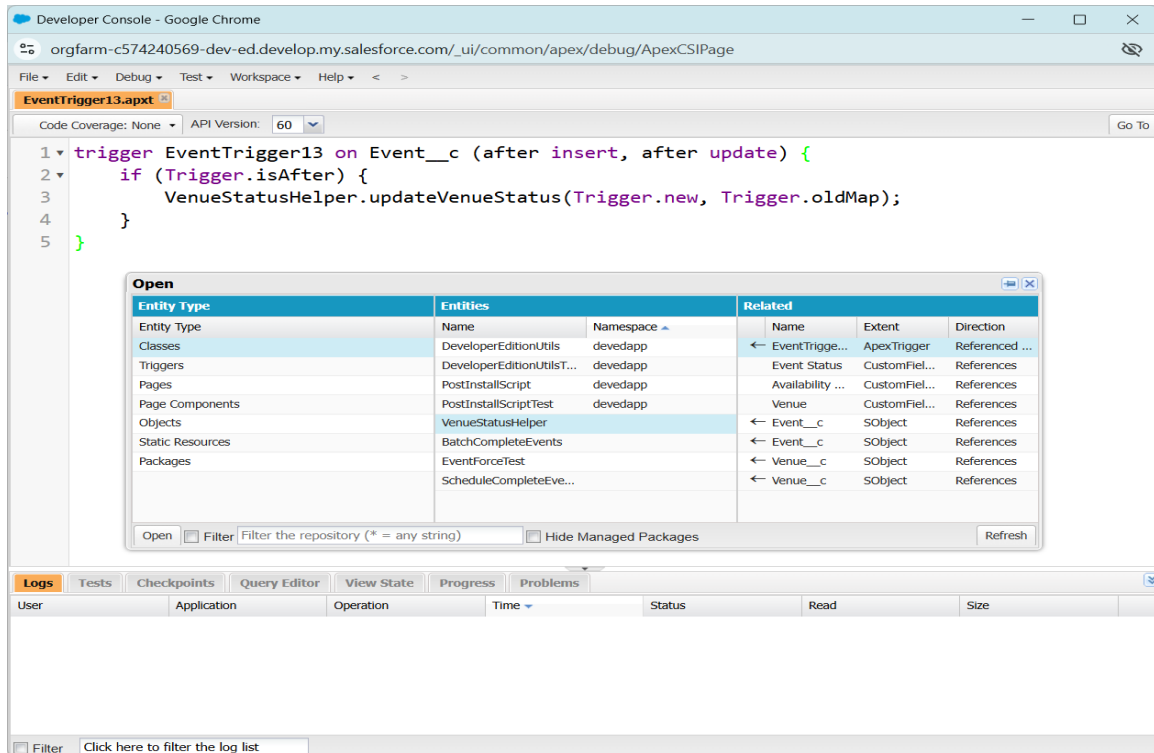
Apex Trigger: EventTrigger13

File → New → Apex Trigger → Name: EventTrigger13, sObject: Event__c:

```

trigger EventTrigger13 on Event__c (after insert, after update) {
    VenueStatusHelper.updateVenueStatus(Trigger.new);
}

```



6.2.8 Milestone 9 – Apex Trigger: Prevent Double Booking

Scenario: two events must not be allowed at the same Venue on the same date. The before-insert/before-update trigger collects the venues involved, queries existing events at those venues in a single query (bulk-safe), and raises an error if another event already exists on the same date.

```

trigger PreventDoubleBooking on Event__c (before insert, before update) {
    Set<Id> venueIds = new Set<Id>();
    for(Event__c ev : Trigger.new){
        if(ev.Venue__c != null) venueIds.add(ev.Venue__c);
    }

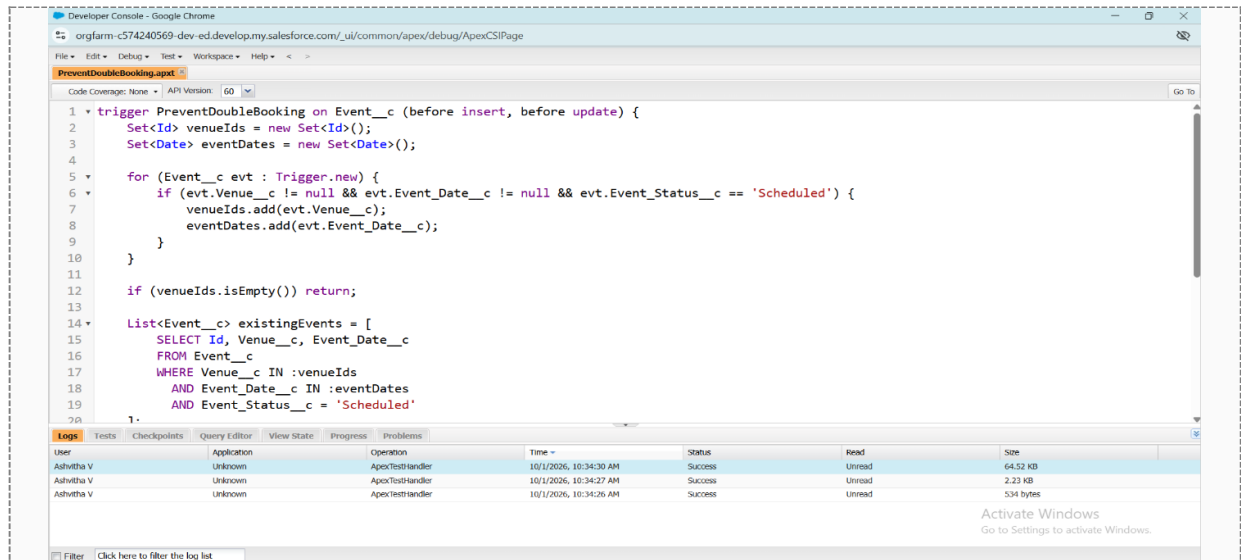
    Map<Id, List<Event__c>> venueEventMap = new Map<Id, List<Event__c>>();
    for(Event__c e : [SELECT Id, Venue__c, Event_Date__c
                      FROM Event__c WHERE Venue__c IN :venueIds]){
        if(!venueEventMap.containsKey(e.Venue__c)){
            venueEventMap.put(e.Venue__c, new List<Event__c>());
        }
        venueEventMap.get(e.Venue__c).add(e);
    }

    for(Event__c ev : Trigger.new){
        if(ev.Venue__c != null && venueEventMap.containsKey(ev.Venue__c)){
            for(Event__c existing : venueEventMap.get(ev.Venue__c)){

```

```

        if(existing.Event_Date__c == ev.Event_Date__c &&
existing.Id != ev.Id){
            ev.addError('This Venue is already booked on this date.');
```



6.2.9 Milestone 10 – Batch & Schedulable Apex

Scenario: mark all past events as Completed every night. A Batch Apex class processes the records in chunks, and a Schedulable class runs the batch automatically.

Batch Class: BatchCompleteEvents

```

global class BatchCompleteEvents implements Database.Batchable<sObject> {
    global Database.QueryLocator start(Database.BatchableContext bc){
        return Database.getQueryLocator(
            'SELECT Id, Event_Date__c, Event_Status__c FROM Event__c ' +
            'WHERE Event_Date__c < TODAY AND Event_Status__c != \'Completed\''
        );
    }
    global void execute(Database.BatchableContext bc, List<Event__c> scope){
        for(Event__c ev : scope){
            ev.Event_Status__c = 'Completed';
        }
        update scope;
    }
    global void finish(Database.BatchableContext bc){
        System.debug('Past events marked as Completed');
    }
}
```

Schedulable Class: ScheduleCompleteEvents

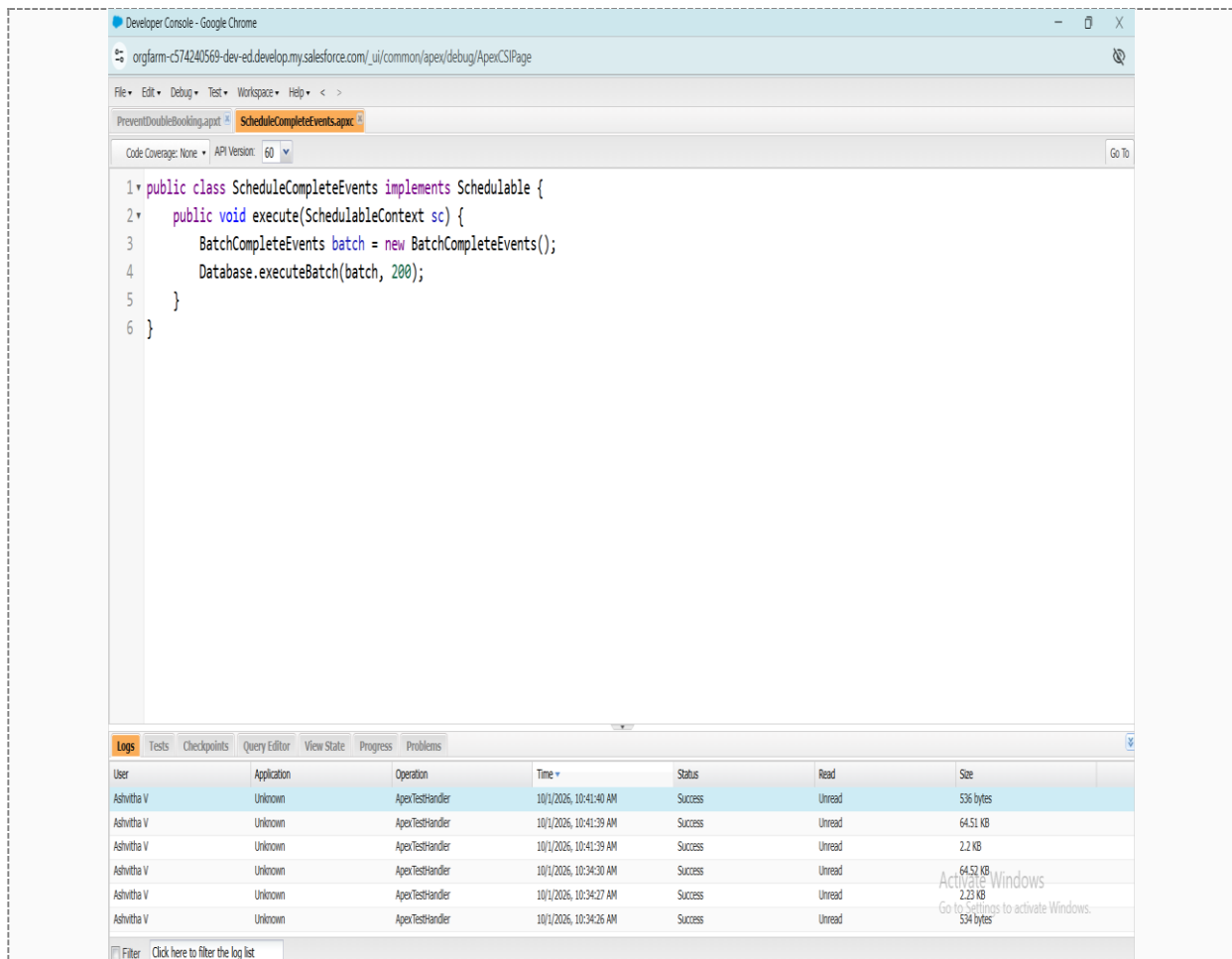
```

global class ScheduleCompleteEvents implements Schedulable {
    global void execute(SchedulableContext sc){
        Database.executeBatch(new BatchCompleteEvents(), 200);
    }
}
```

Scheduling the Job

1. Setup → Apex Classes → click Schedule Apex.
2. Job Name: Daily Event Completion; Apex Class: ScheduleCompleteEvents.

3. Frequency: Weekly; Days: select all days; Preferred Start Time: 08:00 PM.
4. Click Save. The batch now runs every night at 8 PM and marks past events as Completed.



6.3 Phase 3: UI/UX Development & Customization

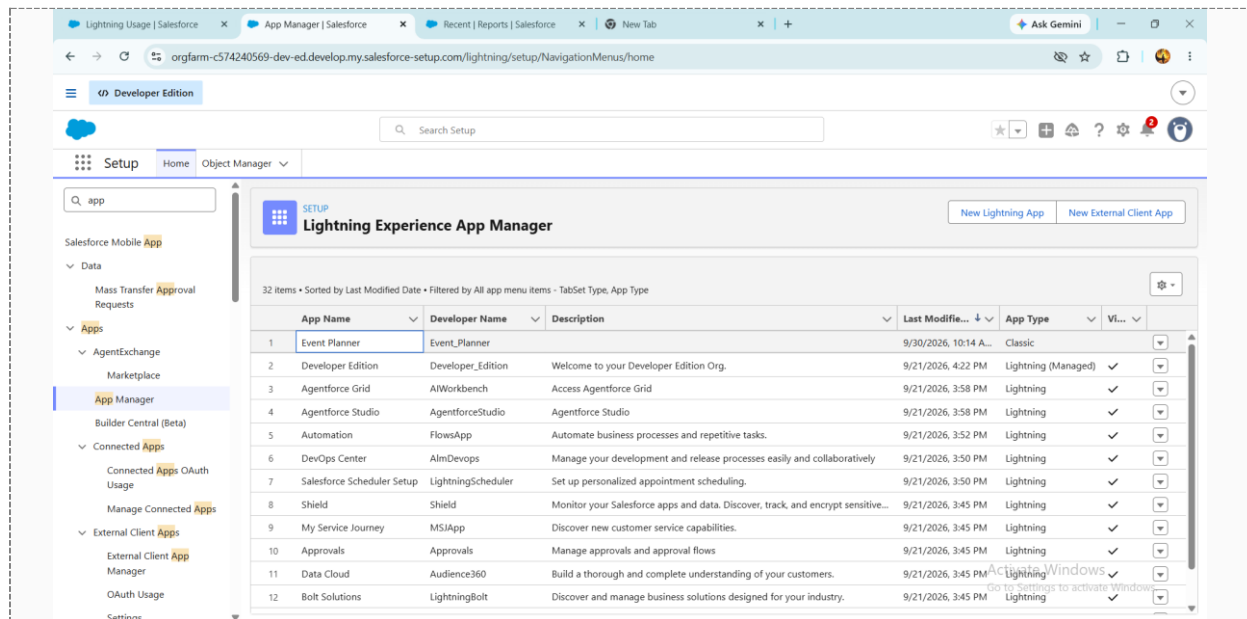
This phase created a simple, user-friendly interface: a custom Lightning App that groups all modules into one workspace, and reports and dashboards that visualize event schedules, vendor performance and client feedback.

6.3.1 Milestone 11 – The Lightning App

An app is a group of tabs that work together as a unit, with a name, a logo and a set of tabs. The Event Planner app was created as follows:

1. Setup → App Manager → New Lightning App. App Name: Event Planner; upload an image; click Next.
2. Leave App Options and Utility Items at their defaults and click Next.
3. From Available Items select Event, Client, Vendor, Venue, Feedback, Reports and Dashboards and move them to Selected Items; click Next.
4. From Available Profiles select System Administrator (and other required profiles) and move them to Selected Profiles.

5. Click Save & Finish. Verify by opening the App Launcher, searching Event Planner and selecting the app.



6.3.2 Milestone 12 – Reports

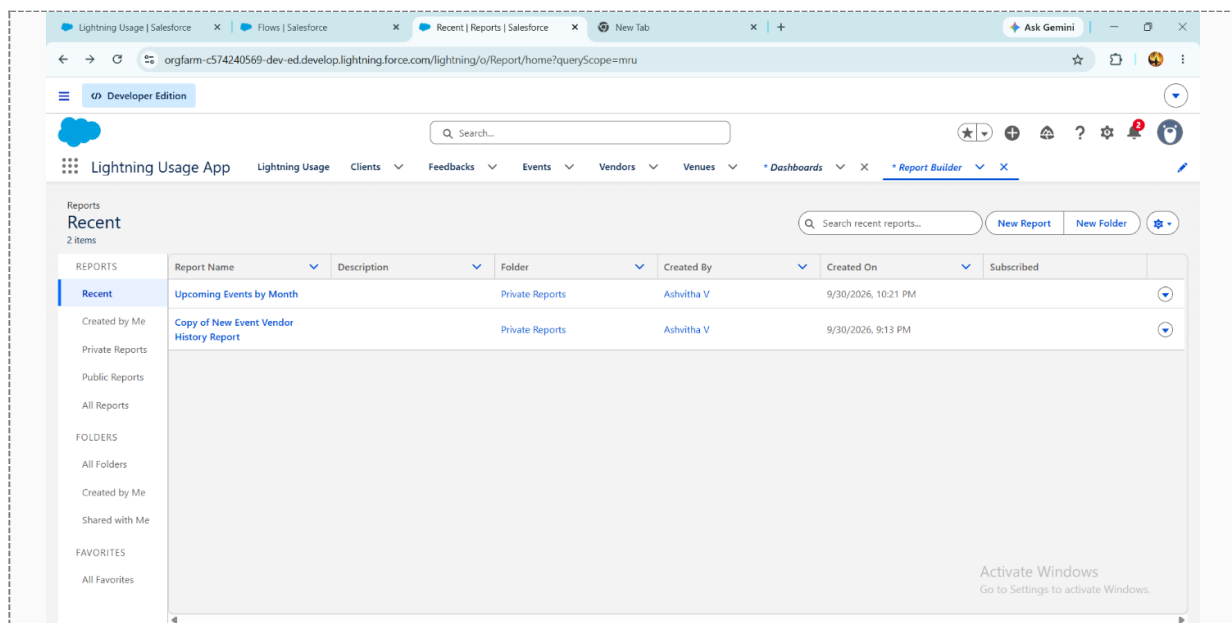
Creating and Sharing the Report Folder

1. App Launcher → Reports → New Folder. Folder Label: EventForce Report; click Save.
2. From All Folders, open the drop-down on the folder → Share.
3. Share with Roles: Event Coordinator and Vendor Manager with View access; click Share, then Done.

Report: Upcoming Events by Month

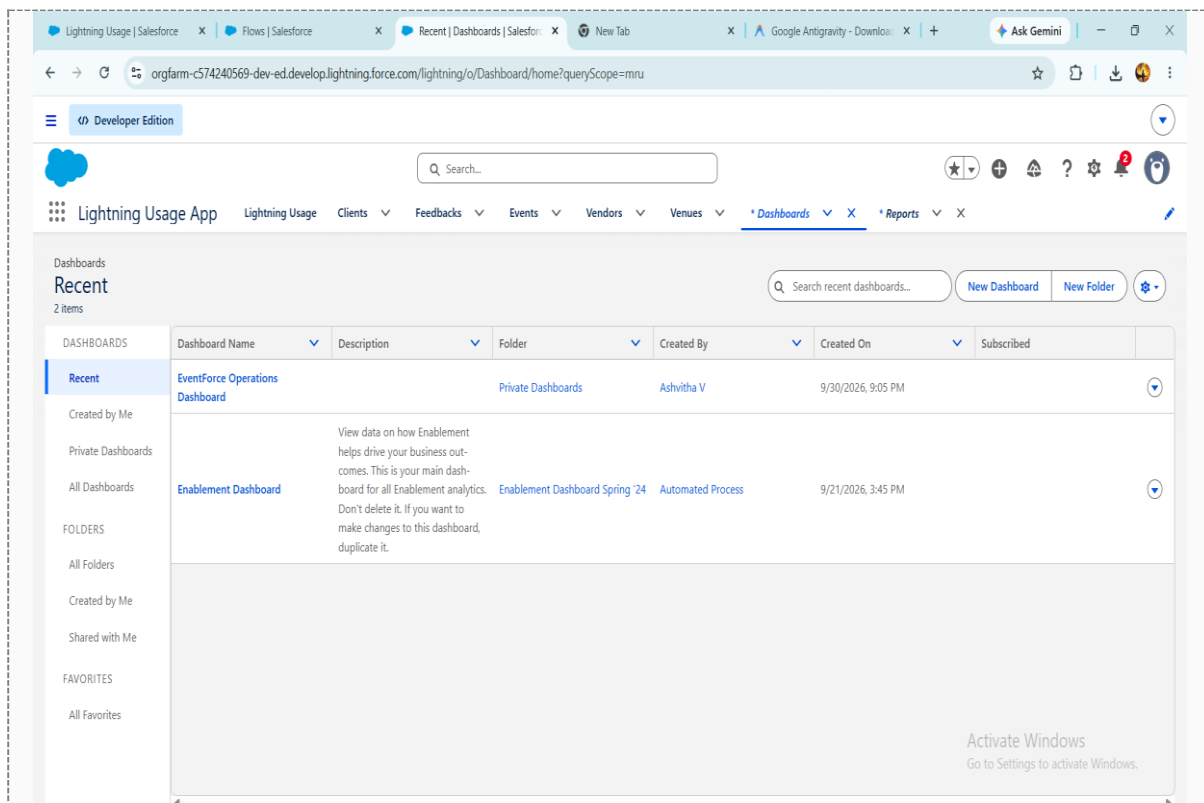
Purpose: view upcoming events and their counts and budgets by month. At least 10 records were created in every object before building the report.

1. Reports → New Report; category All; select Events (or Events with Client & Venue); Start Report.
2. Columns: Event Name, Event Date, Event Type, Client Name, Venue Name, Event Budget.
3. Filters: Show Me = All Events; Event Date ≥ TODAY (or Next 30 Days); Event Status ≠ Canceled.
4. Group Rows: Event Date → Group by Calendar Month.
5. Add Summary: Row Count and SUM of Event Budget.
6. Add Chart: Column or Line; X-axis = Event Date (Calendar Month); Y-axis = Row Count or SUM(Event Budget).
7. Save as Upcoming Events by Month in the EventForce Report folder and click Run.



6.3.3 Milestone 13 – Dashboards

1. Open the Dashboards tab in the Event Planner app → New Dashboard.
2. Name: EventForce Operations Dashboard; click Create.
3. Click + Widget and select the Upcoming Events by Month report.
4. Chart: Donut; Max Values Displayed: 6; Subtitle: Upcoming Events Overview; Widget Theme: Light.
5. Click Save.



6.4 Phase 4: Data Migration, Testing & Security

This phase prepared the system for use by loading validated data, creating users and implementing role-based access control. Records for Events, Clients, Vendors, Venues and Feedback were imported with the Data Import Wizard after pre-validation checks to avoid duplicates. Four types of user accounts (Event Admin, Event Coordinator, Vendor Manager and Client) were created with profiles and roles aligned to their access needs. Organization-Wide Defaults were set to Private and extended by sharing rules.

6.4.1 Milestone 14 – Profiles

A profile is a collection of settings and permissions that define what a user can do in Salesforce (object and field permissions, tab and app settings, login hours, etc.). A user has exactly one profile, but a profile can be assigned to many users.

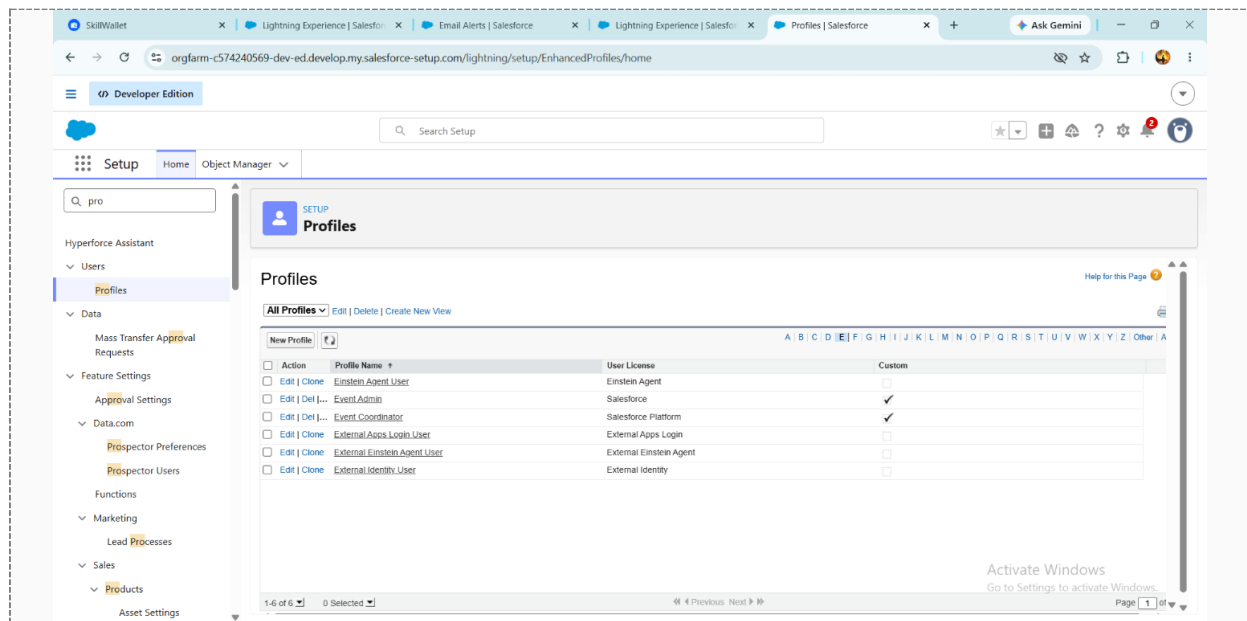
Event Admin Profile

Created by cloning System Administrator (Setup → Profiles → New Profile → Existing Profile: System Administrator → Profile Name: Event Admin). Cloning provides Modify All Data, Customize Application and full CRUD access on all objects, which suits the administrator role.

Custom Profiles for Other Users

The Event Coordinator, Vendor Manager and Client Profile were each cloned from Standard Platform User, then edited: Custom App Settings → default app set to Event Planner; Custom Object Permissions set as below; session timeout set to 2 hours of inactivity; password never expires; minimum password length 8.

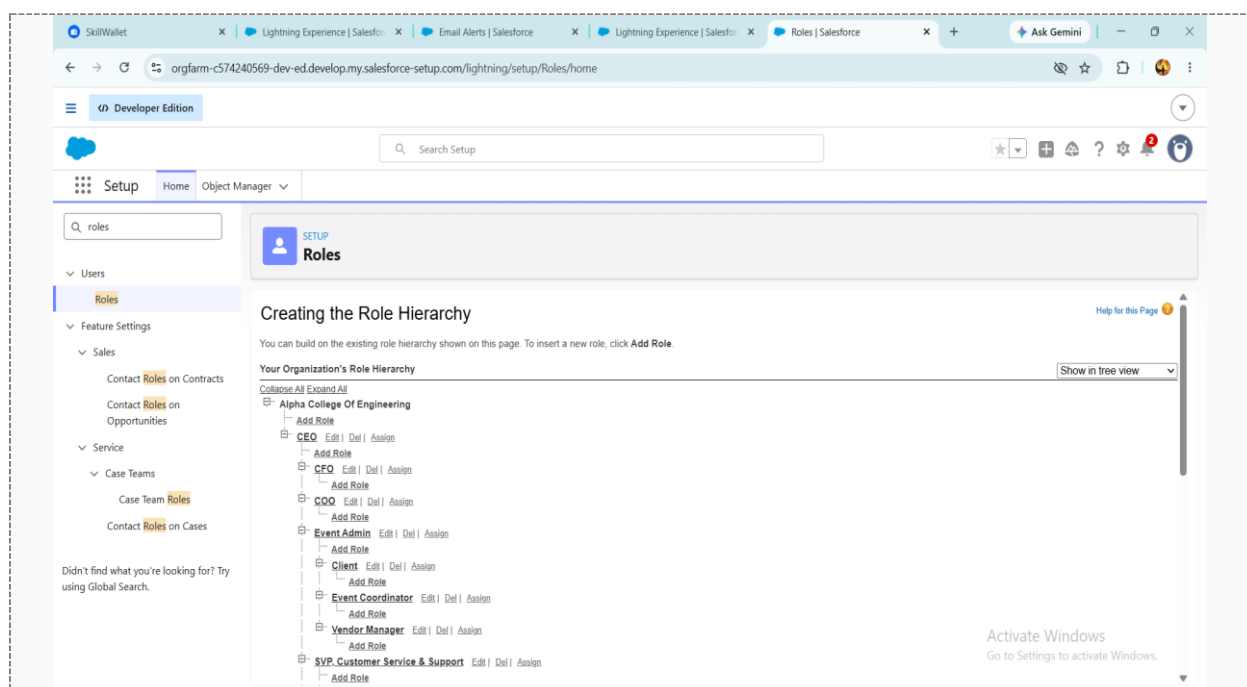
Object	Event Coordinator	Vendor Manager	Client Profile
Event	Create, Read, Edit, Delete	Read only	Read only (own events)
Client	Read, Edit	Read only	—
Vendor	Read	Create, Read, Edit, Delete	No access
Venue	Read	Read only	No access
Feedback	Create, Read	Read only	Create, Read



6.4.2 Milestone 15 – Roles

Roles define the reporting hierarchy and determine record visibility upward through the hierarchy. The Event Admin role was created under CEO (Setup → Roles → Set Up Roles → Expand All → Add Role), and three roles were added beneath it.

Role	Reports To
Event Admin	CEO
Event Coordinator	Event Admin
Vendor Manager	Event Admin
Client	Event Admin



6.4.3 Milestone 16 – Users

A user is anyone who logs in to Salesforce. Users were created from Setup → Users → New User. The first user was created with Role Event Admin, License Salesforce and Profile Event Admin. The other users were created with License Salesforce Platform and the matching Role and Profile.

User Type	Role	User License	Profile
Event Admin	Event Admin	Salesforce	Event Admin
Event Coordinator	Event Coordinator	Salesforce Platform	Event Coordinator
Vendor Manager	Vendor Manager	Salesforce Platform	Vendor Manager
Client	Client	Salesforce Platform	Client Profile

Note: a Developer Edition org has a limited number of Salesforce licenses. If none are available, deactivate an existing user holding that license or change the license type.

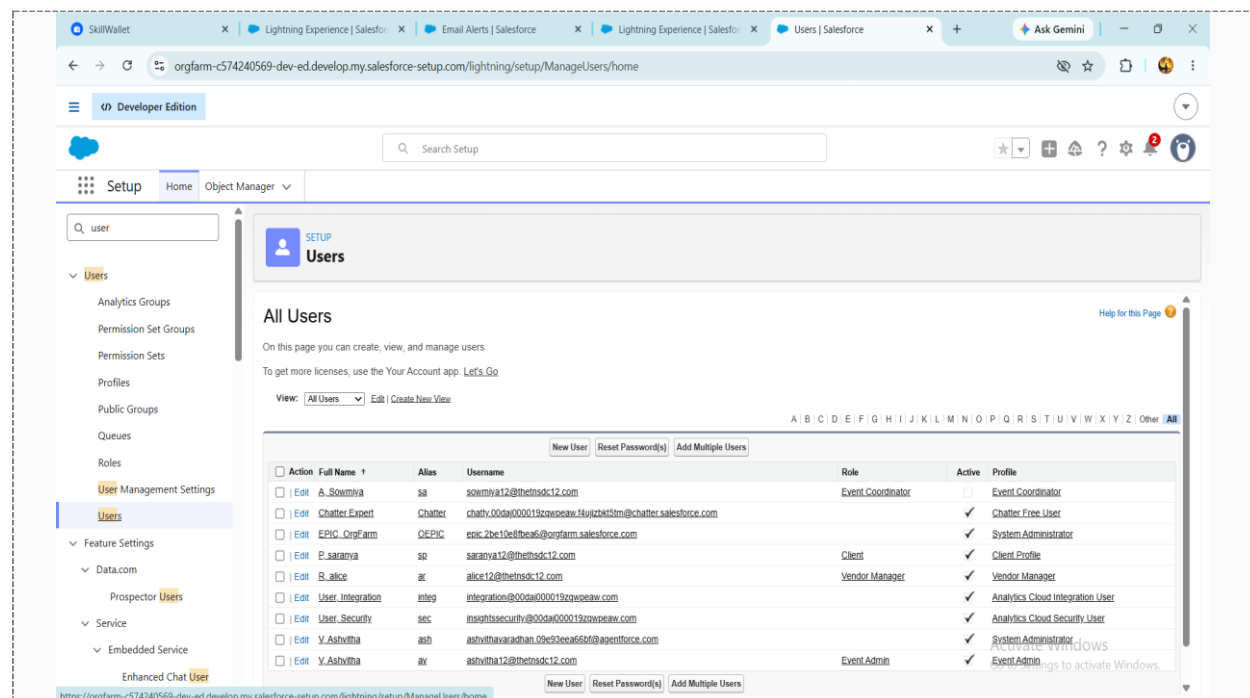
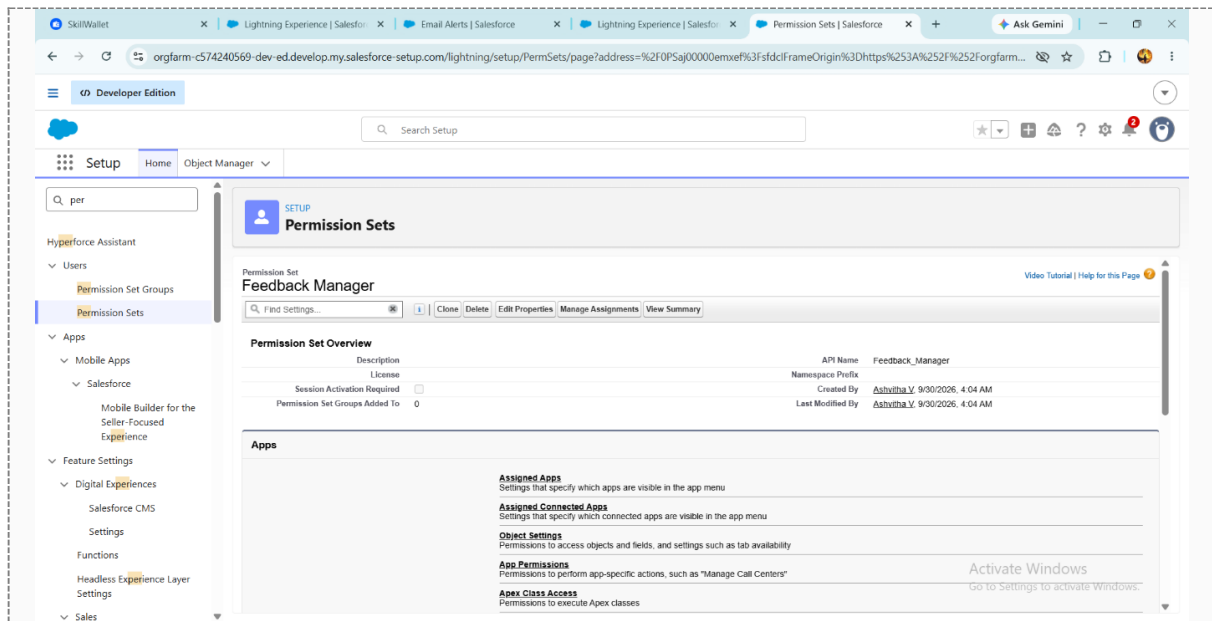


Figure 6.2 – Users

6.4.4 Milestone 17 – Permission Set

A permission set grants extra permissions without changing a user's profile. A Feedback Manager permission set was created to give selected users full access to the Feedback object.

1. Setup → Permission Sets → New. Label: Feedback Manager; click Save.
2. Open Object Settings → Feedback → Edit; enable Read, Create, Edit and Delete; click Save.
3. Click Manage Assignments → Add Assignment, select the Event Coordinator user, and click Next → Assign.



6.4.5 Milestone 18 – Sharing Settings

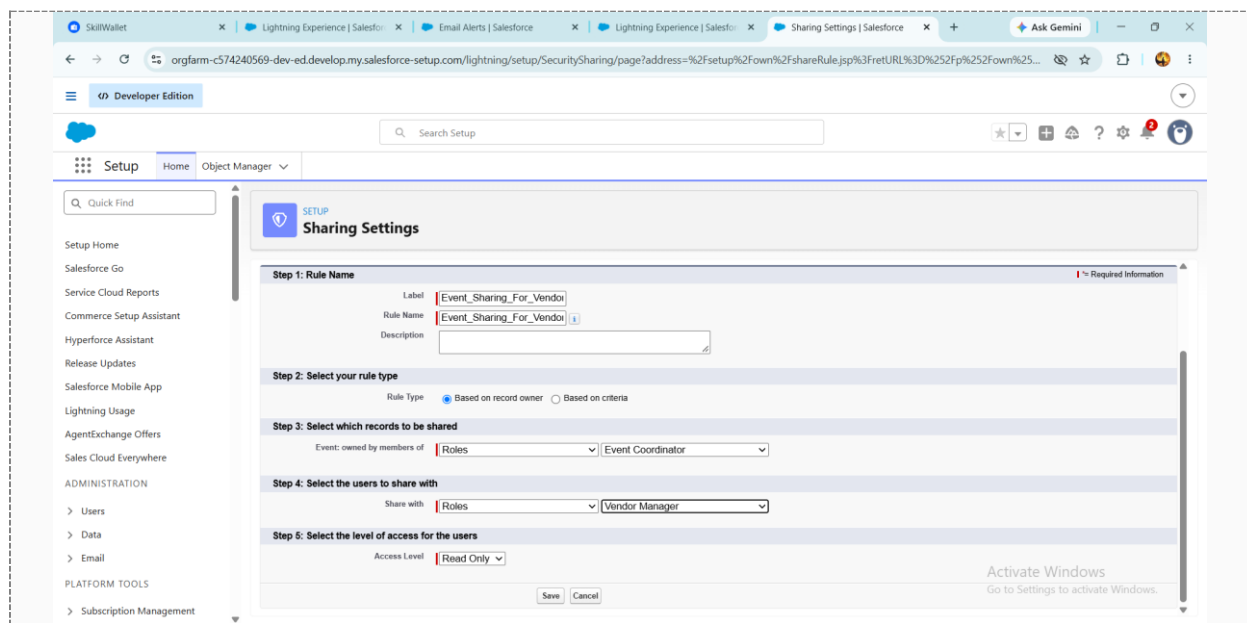
Organization-Wide Defaults (OWD) set the baseline access to records, which can then be opened up using the role hierarchy, sharing rules, manual sharing or Apex sharing. Sharing rules can only extend access, never restrict it.

Configuring OWD

1. Setup → Sharing Settings → Edit (Organization-Wide Defaults).
2. For the Event object set Default Internal Access = Private and Default External Access = Private.
3. Click Save.

Sharing Rule: Event_Sharing_For_Vendors

1. In Sharing Settings find the Event object's Sharing Rules and click New.
2. Label: Event_Sharing_For_Vendors (rule name auto-populates).
3. Records to be shared: members of Roles → Event Coordinator.
4. Share with: Roles → Vendor Manager.
5. Access Level: Read Only; click Save.



6.5 Phase 5: Deployment, Documentation & Maintenance

6.5.1 Deployment Strategy

All configurations, customizations and automations were built and tested within a Salesforce Developer Edition Org. The objective of this phase is to understand how a real-world Salesforce CRM solution is prepared for deployment after configuration, testing and security setup. Actual production deployment is not performed in this project because Developer Edition orgs are standalone and not connected to production instances, and real deployments require sandboxes, change sets or DevOps tooling.

In an enterprise setting, the recommended deployment path would be:

1. Develop and unit-test in a Developer or Developer Pro sandbox.
2. Promote the metadata (objects, fields, flows, Apex, approval process, reports, dashboards) through a Partial/Full sandbox for UAT using Change Sets or Salesforce DX / Git-based CI/CD pipelines.
3. Run Apex tests (minimum 75% code coverage is required for production deployment) and validate the deployment.
4. Deploy to production during a low-usage window, activate flows, approval processes and scheduled jobs, and assign permission sets.
5. Load production data, train users and provide post-go-live support.

Note: Apex test classes were not part of this academic build. Before any production deployment, test classes for VenueStatusHelper, PreventDoubleBooking, BatchCompleteEvents and ScheduleCompleteEvents must be written to meet the 75% coverage requirement.

6.5.2 Maintenance, Monitoring & Troubleshooting

Maintenance keeps event, client, vendor and feedback data accurate and the automations healthy.

Activity	Approach
Monitoring	Regularly review Flow error emails, Apex Jobs and Scheduled Jobs, and pending approvals
Access issues	Resolve user-access errors and permission conflicts by reviewing profiles, permission sets, roles and sharing rules
Debugging	Use Debug Logs and the Developer Console to debug Apex triggers and Flow logic
Rule tuning	Adjust validation rules and sharing settings as business needs evolve
Data quality	Run duplicate checks and periodic data cleansing; review reports for anomalies
Releases	Review Salesforce seasonal release notes and re-test critical automations after each release

Common Issues and Resolutions

Issue	Likely Cause	Resolution
User cannot see the Event Planner app or a tab	Profile or permission set lacks app/tab access	Enable the app/tab visibility in the profile or permission set
Approval process cannot be submitted	No approver/manager configured, or entry criteria not met	Assign a manager on the User record; confirm status is Pending Cancellation
Reminder email not sent	Flow inactive, status not Confirmed, or client email blank	Activate the flow; verify the status and client email
Save blocked on an event	Double-booking trigger fired	Choose a different date or venue
Unable to create a user	No free Salesforce license in the Developer Org	Deactivate an unused user or change the license type

6.5.3 Documentation Summary

This document records the purpose, data model, configuration, automation, security and testing of the EventForce Management System. It covers:

- Business overview and objectives.
- Phase-wise implementation: Requirement → Development → UI → Testing → Security.
- Data model and relationships (Event, Client, Vendor, Venue, Feedback, Event Vendor).
- Automation components: Flow, Approval Process, Validation Rule and Apex logic.
- Security model: Profiles, Roles, Permission Sets, OWD and Sharing Rules.
- Testing results and screenshots of all Salesforce features.

It provides a blueprint for future developers and administrators, supporting consistent development, training and smooth knowledge transfer.

7. FUNCTIONAL & PERFORMANCE TESTING

During testing, screenshots were captured from Salesforce to validate event creation, venue assignment, double-booking prevention triggers, approval status updates, batch processing, scheduled reminders, and dashboard analytics. All features were verified to ensure correct functionality and performance.

Testing validated data accuracy, automation performance and user permissions. The main test scenarios are summarized below.

#	Area	Test Scenario	Expected Result
1	Validation Rule	Save a Client with an invalid email (e.g. “abc@”)	Save blocked with “Please Enter Valid Email Address”
2	Formula Field	Create events of each Event Type	Event Budget auto-populates as per formula
3	Double Booking	Create two events at the same Venue on the same date	Error: “This Venue is already booked on this date.”
4	Venue Status	Set an event to Confirmed, then Canceled	Venue becomes Reserved, then Available
5	Approval Process	Set Event Status to Pending Cancellation and submit	Approval request and alert emails are sent
6	Approval – Approve	Approver approves the request	Status = Canceled; owner notified
7	Approval – Reject	Approver rejects the request	Status = Rejected; owner notified
8	Flow	Confirm an event dated 3+ days ahead with a client email	Reminder email is sent three days before the event date
9	Batch Apex	Run the batch with past-dated events	Past events are marked Completed
10	Lookup Filter	Select a Client on Feedback, then open the Event lookup	Only that client’s events are listed
11	Security	Log in as each user type	Each user sees only the tabs, objects and records permitted
12	Sharing Rule	Vendor Manager views coordinator-owned events	Events are visible as Read Only

8. ADVANTAGES & DISADVANTAGES

Advantages

- **Centralized Data Management** – Event, Client, Vendor, Venue, Feedback, and EventVendor information is managed within Salesforce.
- **Process Automation** – Flows automate reminders and other business processes, reducing manual effort.
- **Prevents Double Booking** – Apex Trigger and VenueStatusHelper validate venue availability and help prevent booking conflicts.
- **Controlled Cancellation Process** – Approval Process provides structured handling of event cancellation requests.
- **Improved Operational Visibility** – Reports and Dashboards provide visibility into event schedules, vendor activities, and feedback.
- **Secure Access** – Profiles, Roles, Permission Sets, OWD, and Sharing Rules provide controlled access to records.
- **Scalable Solution** – Salesforce provides a platform that can support future growth and additional business requirements.
- **Better Data Quality** – Validation Rules and pre-validation during data migration help maintain accurate and consistent data.

Disadvantages

- **Developer Edition Limitation** – The project was implemented in a Salesforce Developer Edition for academic purposes and was not connected to a production environment.
- **Production Deployment Not Performed** – Actual migration to production was outside the scope of this implementation.
- **Deployment Requires Additional Tools** – Real-world deployment may require sandboxes, Change Sets, Salesforce DX, or DevOps/CI-CD tools.
- **Configuration Complexity** – Managing Flows, Apex, Approval Processes, and security configurations requires appropriate Salesforce knowledge.
- **Ongoing Maintenance Required** – Flows, Apex jobs, approval processes, validation rules, and sharing settings need regular monitoring and troubleshooting.

9. CONCLUSION

The EventForce Management System successfully demonstrates how Salesforce can be used to streamline and automate event planning and management operations. The project brings Events, Clients, Vendors, Venues, Feedback, and EventVendor data into a centralized platform.

The implementation combines Salesforce Lightning, Flows, Validation Rules, Formula Fields, Approval Processes, Apex Triggers, Apex Classes, Batch Apex, Scheduled Apex, security controls, Reports, and Dashboards to address key business requirements. Automation supports activities such as event reminders, cancellation approvals, venue availability management, double-booking prevention, and event completion.

The project also demonstrates the importance of data migration, testing, security, documentation, and maintenance in a Salesforce implementation. Although the solution was developed and tested in a Developer Edition Org and actual production deployment was not performed, the project reflects real-world Salesforce implementation practices and provides a foundation for future scalability and enhancement.

Key Learnings

- Designing a scalable data model, including junction objects for many-to-many relationships.
- Combining declarative tools (Flows, Approval Processes, Validation Rules) with Apex for advanced logic.
- Writing bulk-safe Apex triggers and using Batch and Schedulable Apex for asynchronous processing.
- Implementing layered security with Profiles, Roles, Permission Sets, OWD and Sharing Rules.
- Building reports and dashboards that turn operational data into decisions.
- Understanding how a Developer Org build maps to real-world deployment, maintenance and documentation practices.

10. FUTURE SCOPE

The EventForce Management System can be further enhanced to support larger-scale event operations and improve automation, integration, and user experience.

- **Production Deployment** – Deploy the solution from development to a production Salesforce environment using appropriate deployment tools.
- **Advanced Event Automation** – Extend automation for additional event planning and operational activities.
- **Enhanced Vendor Management** – Introduce more advanced vendor tracking and performance management capabilities.
- **Advanced Analytics** – Add more reports and dashboards for deeper insights into events, venues, vendors, and client feedback.
- **External Integrations** – Integrate Salesforce with external systems and services required for real-world event operations.
- **Mobile Accessibility** – Enhance the solution for convenient event management through mobile devices.
- **Scalability Enhancements** – Expand the system to support larger numbers of events, clients, vendors, venues, and users.
- **Continuous Maintenance & Optimization** – Regularly improve Flows, Apex, validation rules, sharing settings, and other configurations as business requirements evolve.

These enhancements can help transform the current EventForce implementation into a more comprehensive and enterprise-ready event management CRM solution.

11. APPENDIX

- **A. Custom Objects:** Event, Client, Vendor, Venue, EventVendor, Feedback
- **B. Automation:** Flows, Validation Rules, Approval Process, Apex Triggers & Classes
- **C. Processing:** Batch Apex and Scheduled Apex
- **D. Security:** Profiles, Roles, Permission Sets, OWD, Sharing Rules
- **E. Data Migration:** Salesforce Data Import Wizard
- **F. UI & Analytics:** Lightning App, Reports & Dashboards
- **G. Testing & Documentation:** Configuration testing, security testing, automation testing, and project documentation

Thank You